

L1VM - Brackets manual

BY STEFAN PIETZONKE

<http://midnight-coding.de>

7. August 2026

V3.2.12

I - Introduction

1 Abstract

The L1VM is a general purpose VM. It is programmed in C and C++. My language Brackets is type safe and has variable overflow checking at run-time. The VM has 256 “int64” registers and 256 “double floating point” registers. The opcodes use the registers to store the data. Most opcodes are of three address kind with the third register as a target. The VM also has a stack to store the registers before a function call. Setting the needed function call arguments on the stack. The called function puts the return values on the stack. The caller gets the return values from the stack. And restores the callers stack after the function call.

“Interpreters designed for high general-purpose performance typically perform a large number of indirect branches (3.2%–13% of all executed instructions in our benchmarks). These branches consume more than half of the run-time in a number of configurations we simulated. We evaluate how accurate various existing and proposed branch prediction schemes are on a number of interpreters, how the mispredictions affect the performance of the interpreters and how two different interpreter implementation techniques perform with various branch predictors. We also suggest various ways in which hardware designers, C compiler writers, and interpreter writers can improve the performance of interpreters.” [1]

The L1VM uses indirect threading as the opcode dispatch method. It is using the “labels as values” feature of gcc and clang. The main advantage of this is the faster opcode dispatch during run-time. The opcode labels are stored in a jump table. There is a macro in the C code of the VM which loads the next opcode, makes a jump table lookup and then branches to the next opcode.

“Some people argue that efficiency is not important in interpreters because they are slow anyway. However, this attitude can lead to an interpreter that is more than a factor of 1000 slower on many programs than native code produced by an optimizing compiler [1], whereas the slowdown for efficient interpreters is only a factor of 10 [3]. I.e., the difference between a slow and a fast interpreter is larger than the difference between a fast interpreter and native code.” [2]

A Mandelbrot fractal rendering program running on L1VM was about factor 7 slower as the native program running in C.

There is a JIT-compiler for x86 and AARCH64 CPU platforms. It can translate the math and jump opcodes (if/else/endif) into assembly code targeting the native CPU. In Brackets functions can be written in inline assembly. This assembly code can be compiled by the JIT-compiler if only the allowed opcodes are used. The VM uses the “AsmJIT” library JIT-compiler.

The VM is a RISC design with about 61 opcodes. There are two special interrupts with build in functions always available. The L1VM uses registers to store the opcode arguments. This is faster than a stack based one. [3]

1. “The Structure and Performance of Efficient Interpreters” (11/2003) - page 1

M. Anton Ertl - Institut für Computersprachen, TU Wien, Argentinestraße 8, A-1040 Wien

David Gregg - Department of Computer Science, Trinity College, Dublin 2, Ireland

2. “The Structure and Performance of Efficient Interpreters” (11/2003) - page 2

3. “Virtual Machine Showdown: Stack Versus Registers” (2008)

Yunhe Shi, David Gregg, Andrew Beatty, Department of Computer Science University of Dublin, Trinity College Dublin 2, Ireland

M. Anton Ertl - Institut für Computersprachen, TU Wien, Argentinestraße 8, A-1040 Wien

Here is the source code of the L1VM jump table with the opcode labels (main.c):

```
// jumptable for indirect threading execution
static void *jumpt[] =
{
    &&pushb, &&pushw, &&pushdw, &&pushqw, &&pushd,
    &&pullb, &&pullw, &&pulldw, &&pullqw, &&pulld,
    &&addi, &&subi, &&muli, &&divi,
    &&addd, &&subd, &&muld, &&divd,
    &&smuli, &&sdivi,
    &&andi, &&ori, &&bandi, &&bori, &&bxori, &&modi,
    &&eqi, &&neqi, &&gri, &&lsi, &&greqi, &&lseqi,
    &&eqd, &&neqd, &&grd, &&lsd, &&greqd, &&lseqd,
    &&jmp, &&jmpi,
    &&stpushb, &&stpopb, &&stpushi, &&stpopi, &&stpushd, &&stpopd,
    &&loada, &&loadd,
    &&intr0, &&intr1, &&inclsijmp, &&decgrijmp,
    &&movi, &&movd, &&loadl, &&jmpa,
    &&jsr, &&jsra, &&rts, &&load,
    &&noti
};
```

The execution macro is this (main.h):

```
#define EXE_NEXT(); ep = ep + eoffs; goto *jumpt[code[ep]];
```

The “eoffs” is the offset from the current “ep” (execution pointer) to the next opcode. It is set by each opcode at the end. Every opcode calls the “EXE_NEXT” macro at the end. There is no main loop in the VM.

Here is the “addi” opcode which adds “int64” numbers:

```
addi:
#ifdef DEBUG
printf ("%lli ADDI\n", cpu_core);
#endif
arg1 = code[ep + 1];
arg2 = code[ep + 2];
arg3 = code[ep + 3];

#ifdef MATH_LIMITS_INT
    if (__builtin_saddll_overflow (regi[arg1], regi[arg2], &regi[arg3]))
    {
        overflow = 1;
        printf ("ERROR: overflow at addi!\n");
        PRINT_EPOS();
    }
    else
    {
        overflow = 0;
    }
#else
    regi[arg3] = regi[arg1] + regi[arg2];
#endif

eoffs = 4;
EXE_NEXT();
```

The registers “arg1” and “arg2” are added and stored in the “arg3” register. If the math limits flag is set then an overflow checking is done.

2 Installation

The L1VM can be installed with the install scripts for the host OS.

1. Set the needed paths and ENV variables in your ".bashrc" file:

```
export PATH="$HOME/l1vm/bin:$PATH"
export LD_LIBRARY_PATH="$HOME/l1vm/bin:$LD_LIBRARY_PATH"
export L1VM_ROOT="$HOME/l1vm/"
```

This sets the "~/l1vm" directory in your "/home" directory as the home for L1VM. Which is the default.

2. Choose one of the build scripts for your OS.

On Debian Linux for example (in the "install" directory):

```
$ ./install-debian.sh
```

The installer scripts need "sudo" to install the needed libraries.

You have to make sure that sudo works on your system!

3 Variables

In Brackets variables are defined by the "set" function:

```
(set int64 1 zero 0)
```

This defines the variable "zero" as an int64 number with the value "0".

The number variable types are: "byte", "int16", "int32", "int64", "double" and "bool".

The "double" variable type is a 64 bit floating point number.

There is also a type "string" for string variables:

```
(set string s hellostr "Hello world!")
```

Here the "s" as the size defines enough space for the string. You could also write:

```
(set string 13 hellostr "Hello world!")
```

So here the size of the string is set to "13". This is one more than the string length.

A string variable is also an array. To allocate an array for "5" int64 numbers this can look like this:

```
(set int64 5 numarray 0 1 77 85 2)
```

3.1 Arrays

Here is a program which shows how to use a number array:

```
// array-demo.llcom
// array with values demo
// shows how to set up an array
// and how to access the elements
//
#include <intr-func.llh>
(main func)
  (set int64 1 zero 0)
  (set int64 1 one 1)
  (set const-int64 1 offset 8)
  (set int64 1 ind 0)
  (set int64 1 f 0)
```

```

(set int64 1 num 0)
(set int64 1 realind 0)
// set array data using spaces as elements separator
(set int64 10 array 10 5 8 4 3 2 7 23 45 30)
(set int64 1 maxarray 10)
// print array
(((ind maxarray <) f =) f for)
  ((ind offset *) realind =)
  (array [ realind ] num =)
  (num :print _i !)
  (:print _n !)
  ((ind one +) ind =)
(next)
(zero :exit !)
(funcend)

```

Here is the output:

```

$ llvm prog/array-demo -q
10
5
8
4
3
2
7
23
45
30

```

The line:

```
((ind offset *) realind =)
```

Multiplies an “offset” of “8” to the number index. This is needed because the variables are all in one big byte array. So on the index “0” there is the first array variable. The next one is at index “8”.

An int64 variable uses 8 bytes in the memory. For the other number types a different “offset” is used.

3.2 Constants

A constant variable which can not be changed at runtime is defined like this:

```
(set const-int64 1 offset 8)
```

There is a special flag which can be set to define “immutable” variables as default:

```

(variable-immutable)

// hello-mut.llcom
#include <intr-func.llh>
(main func)
  // all default variables are immutable after this is set!
  (variable-immutable)
  // immutable:
  (set int64 1 zero 0)
  (set int64 1 x 23)
  (set int64 1 y 42)
  // mutable:
  (set mut-int64 1 a 0)
  (set string s hello "Hello world!")
  // print string
  (hello :print _s !)

```

```

    (:print_n !)
    ((x y *) a =)
    (a :print_i !)
    (:print_n !)
    (zero :exit !)
(funcend)

```

3.3 Boolean variables

A bool variable can be defined like this:

```
(set bool 1 Bool 1)
```

The variable name of a bool variable must begin with an uppercase “B”. As an value only “0”, “1”, “false” and “true” can be used.

```

// bool.llcom
// set boolean variable checks by using "bool" command
// only true or false allowed!
#include <intr-func.llh>
#include <bool.llh>
(main func)
    (set int64 1 zero 0)
    (set int64 1 one 1)
    (set int64 1 a 42)
    (set int64 1 b 23)
    (set int64 1 c 0)
    (set int64 1 f 0)
    // boolean variable name must beginn with upercase B!
    (set bool 1 Bool 1)
    (set byte 1 bvar 23)
    (set string s boolstr "Bool = ")
    (a c =)
    // set Bool to true if number = 42
    (((c a ==) f =) f if+)
    (true Bool =)
    (else)
    (false Bool =)
    (endif)
    (boolstr :print_s !)
    (Bool :print_i !)
    (:print_n !)
    (zero :exit !)
(funcend)

```

Here is the output of the program:

```

$ llvm prog/bool -q
Bool = 1

```

3.4 Local variables

In “intr-func.llh” four function macros are defined:

```

#func data_func_local_alloc (NONE) :(11 0 0 0 intr1)
#func data_func_local (NONE) :(12 0 0 0 intr1)
#func data_func_global (NONE) :(13 0 0 0 intr1)
#func data_func_free (NONE) :(14 0 0 0 intr1)

```

They can be called like this:

```
(:data_func_local_alloc !)
```

This allocates a new variable data space. And copies all variables into it.

With “data_func_local” only the local variables are used. The global variables are not modified!

To switch back to the normal global variables use “data_func_global”.

To free the memory call “data_func_free”.

The local variables are useful if the program has threads running. So the local variables of the thread are used isolated from each other. A full example program is in “prog/webserver-thread-local-2.llcom”. It is a simple webserver which runs multiple threads for each client.

3.5 Variable prefixes

In Brackets variable prefixes can be set by backslashes:

```
(set int64 1 \distance\x 2)
```

Here the prefix is “distance”. This can be used in math expressions. If the target variable has a prefix then all other variables in the expression must have the same type.

This example does not compile:

```
// type.llcom
// Brackets - number prefixes \prefix\
// only variables with the target var prefix allowed in math expressions
//
#include <intr-func.llh>
(main func)
    (set int64 1 zero 0)
    (set int64 1 \distance\x 2)
    (set int64 1 y 10)
    (set int64 1 \distance\z 0)
    // gives compiler error, because y is not of type \distance\ !
    (\distance\x + y \distance\z :=)
    (\distance\z :print_i !)
    (:print_n !)
    (zero :exit !)
(funcend)

$ llvm-build.sh type
building: type
error: check_variable_prefix: prefix must be: '\distance\' on: 'y'!
> (\distance\x + y \distance\z :=)

error: check_variable_prefix: prefix must be: '\distance\' on: 'zero'!
> (255 zero 0 0 intr0)

code lines compiled: 102
ERRORS! can't read source file!
[!] out

build failed!
```

Here the second error message about “zero” can be ignored! You have to rename “y” to use the “distance” prefix:

```
(set int64 1 \distance\y 10)
```

4 The new linter

I developed a linter for my Brackets language for my L1VM. You can use it to declare the function call arguments variable types. And the return variable types can be defined too.

Both is declared in a special comment form. The linter runs before the compiler. The compiler does ignore the linter comments. To use the linter you have to build it first in the linter directory:

```
$ CC=clang ./make.sh
```

Now the linter should be build. You can also use the “gcc” compiler to build it.

Copy the linter in to your “~/llvm/bin” directory (in your home directory). To use it call the new build script:

```
“llvm-build-lint.sh”.
```

The linter is forced by the linter flag in the code:

```
(linter)
```

4.1 Set the function call variable types

```
// (func args add int64 int64)
```

This declares that the function “add” has two variables of type int64 as arguments.

4.2 Set the function return variable types

To set the return values we can write this:

```
// (return args add int64)
```

This declares that the function “add” has one return value of type int64.

If you do a call of the function add, then the linter will check if the variable types and the number of the variables do match:

```
(x~ y~ :add !) (sum~ stpop)
```

And here you will see something new: the “stopop” command is in the same line as the function call.

You have to write the call and the stack pop of the return values this way!

The variable types of the linter are:

```
int64, double, byte, any and variable
```

With “any” every variable type matches. This is useful if a function can handle more than one variable type. The “variable” type sets that the number of variables and their type can change.

4.3 Control the linter

You can turn the linter on and off by:

```
// lint-on  
// lint-off
```

This can be useful when you want to switch it temporarily off in some cases.

5 Set legal ranges for variables

You can set the minimum and maximum value for a number variable.

The VM does runtime checks and returns an exception if a variable is in illegal range.

5.1 An example

```
(set int64 1 sum~ 0)
(set const-int64 1 sum_min~ 0)
(set const-int64 1 sum_max~ 1000000)

// set legal range for sum variable:
(sum~ sum_min~ sum_max~ range)
```

So here is the minimum value set to “0” and the maximum value to “1000000”.

If at runtime the value of the variable “sum~” is out of range, then the VM will break with an exception.

6 Putting it all together

Here I will show a simple demo program with the new linter feature and the variable ranges check.

6.1 The demo program

Here is the demo program “hello-add-range.llcom”:

```
// hello-add-range.llcom - with the new linter args
#include <intr-func.llh>
#include <zero.llh>
#include <misc-macros.llh>
// this is the function declaration for the llvm-linter:
// it has two int64 numbers as arguments:
//
// (func args add int64 int64)
//
// set the return variable type
//
// (return args add int64)
//
(main func)
  #var ~ main

  (set int64 1 zero 0)
  (set int64 1 x~ 23)
  (set int64 1 y~ -30)
  (set int64 1 sum~ 0)
  (set const-int64 1 sum_min~ 0)
  (set const-int64 1 sum_max~ 1000000)

  // set legal range for sum variable:
  (sum~ sum_min~ sum_max~ range)

  (x~ y~ :add !) (sum~ stpop)
  // pull the function return value in sum~ and do the range check
  (sum~ :pull_int64_var !)
  (sum~ :print_i !)
  (:print_n !)

  (zero :exit !)
(funcend)
(add func)
  #var ~ add

  (set int64 1 x~ 0)
  (set int64 1 y~ 0)
```



```

(set int64 1 ret~ 0)

(y~ x~ stpop)
(x~ + y~ ret~ :=)
(ret~ stpushi)
(funcend)

```

6.2 Run the demo program

You have to build the demo first:

```

$ llvm-build-lint.sh hello-add-range
building: hello-add-range
second include path: '/home/stefan/llvm-work/prog/linter/'
llvm-linter 3.0.9 , no errors found!
code lines compiled: 32
[✓] out compiled
assembler args: ' '
assembling file: 'out'
codesize: 361 bytes
datasize: 669 bytes
filesize: 1155 , 1.128 KB
[✓] out assembled

build in 0.0040 seconds
copying files...

```

And now run it:

```

$ llvm hello-add-range -q
ERROR: int variable value in illegal range!
var: -7 : min: 0, max: 1000000

epos: 189

```

Now you see the runtime exception because the variable “sum~” is now “-7”!

The program added “23 + -30” = “-7”. So the variable “sum~” is out of range.

In this example I used the “pull_int” function of the Brackets “misc.macros.llh” include file:

```
pull_int64_var (sum~)
```

Here is the “misc-macros.llh” include file:

```

>> misc-macros.llh
>> some useful macros
>> variable pull macros, add zero to store register into variable
>> You must include "zero.llh" first!
#func pull_byte_var (VAR) :{VAR = VAR + zero_byte}
#func pull_int16_var (VAR) :{VAR = VAR + zero_int16}
#func pull_int32_var (VAR) :{VAR = VAR + zero_int32}
#func pull_int64_var (VAR) :{VAR = VAR + zero}
#func pull_double_var (VAR) :{VAR = VAR + zero_double}
>> inc and dec integers
#func inc (VAR) :((VAR one +) VAR =)
#func dec (VAR) :((VAR one -) VAR =)

```

Here is the definition:

```
#func pull_int64_var (VAR) :{VAR = VAR + zero}
```

This just adds zero to an variable and stores it into the variable back.

It is needed to trigger the range out of bounds check here. Without this “pull_int” macro the return value of the function “add” would not be checked! Keep this in mind!

If you start the debugger you will see this execution position “189” assembly listing:

```
$ llvm-debug.sh 189
47
intr0, 28, 3, 4, 5
stpopi 0
loada summain, 0, 1
intr0 4, 1, 0, 0
intr0 7, 0, 0, 0
loada zero, 0, 2
intr0 255, 2, 0, 0
rts
:add
loada zero, 0, 0
stpopi 1
load yadd, 0, 2
pullqw 1, 2, 0
stpopi 2
load xadd, 0, 3
pullqw 2, 3, 0
addi 2, 1, 3
load retadd, 0, 4
```

You can see the variable “summain” in the assembly listing above. This is the assembly code generated by the Brackets compiler.

7 Pure functions

Now you can mark functions as “pure”. This means that the function returns the same values if it is called with the same input. And it is independent of global state.

7.1 Example code

```
(squareP func)
  #var ~ square)
  (set double 1 num~ 0.0)
  (set double 1 ret~ 0.0)
  (num~ stpopd)
  (num~ * num~ ret~ :=)
  (ret~ stpushd)
(funcend)
```

The function name ends with an uppercase “P” to mark it as a pure function. You can only call other pure functions from this function type! To switch this off you can use:

```
(pure-off)
```

8 Documentation

You can output program documentation in to a markdown file by setting documentation blocks in your code. The start is marked by “#docustart” and the end by “#docuend”.

8.1 An example

Here is an example (from the “math-circle-bignum.llcom” program:

```
(circle object)
```

```

#var ~ circle

#docustart
circle object
=====
Calculate diameter, circumference and area of a circle.

init->circle
-----
Set the m_pi Pi variable.

calc_diam->circle
-----
Calculate the diameter of a circle.

calc_circ->circle
-----
Calculate the circumference of a circle.

calc-area->circle
-----
Calculate the area of a circle.
#docuend

```

9 Variable scope

You can set the variable scope by some flags.

9.1 The flags

You can switch on to access only local and global variables only by:

```
(variable-local-on).
```

After this only the function local and global variables are allowed to access. The global variables must end by “main” in the name. To switch this off you can use:

```
(variable-local-off).
```

You can set a flag to allow only the function local variables with:

```
(variable-local-only-on).
```

10 Variable handling

You can set the variable settings with two flags.

10.1 The flags

```
(variable-immutable)
(variable-mutable)
```

The “variable-immutable” flag sets every new declared variable as immutable (not changeable). This is the same as with the “const” beginning for variables. You can switch this off by “variable-mutable” flag.

11 Variable definition

In Brackets all variables are global variables. The local variables in functions are defined with the unique “~” ending.

```
#var ~ foo
```

```
(set int64 1 x~ 42)
```

This defines the variable name “xfoo”. The “~” is set as “foo” in:

```
#var ~ foo
```

```
(square func)
```

```
  #var ~ square
```

```
    (set int64 1 num~ 0)
```

```
    (set int64 1 square~ 0)
```

```
      (num~ stpop)
```

```
      (num~ * num~ square~ :=)
```

```
      (square~ stpush)
```

```
(funcend)
```

11.1 Binary and hex numbers

Binary numbers can be defined with a beginning dollar sign:

```
(set byte 1 bbin B$10000001)
```

And hex numbers by a beginning “&”.

```
// binary-2.llcom - Brackets - binary demo ;)
```

```
#include <intr-func.llh>
```

```
(main func)
```

```
  (set int64 1 zero 0)
```

```
  // HEX numbers
```

```
  (set byte 1 bhex &FF)
```

```
  (set int16 1 i16hex &FFFF)
```

```
  (set int32 1 i32hex &FFFFFFFF)
```

```
  (set int64 1 i64hex &FFFFFFFFFFFFFFFF)
```

```
  // binary numbers
```

```
  (set byte 1 bbin B$10000001)
```

```
  (set int16 1 i16bin W$10000000000000001)
```

```
  // normal number
```

```
  (set int64 1 i64norm 1234567890123456)
```

```
  (bhex :print_i !)
```

```
  (:print_n !)
```

```
  (i16hex :print_i !)
```

```
  (:print_n !)
```

```
  (i32hex :print_i !)
```

```
  (:print_n !)
```

```
  (i64hex :print_i !)
```

```
  (:print_n !)
```

```
  (:print_n !)
```

```
  (bbin :print_i !)
```

```
  (:print_n !)
```

```
  (i16bin :print_i !)
```

```
  (:print_n !)
```

```
  (i64norm :print_i !)
```

```
  (:print_n !)
```

```
  (zero :exit !)
```

```
(funcend)
```

And this is the output:

```
255
65535
4294967295
9223372036854775807
```

```
129
32769
1234567890123456
```

The flags are:

B = byte, W = word, D = double word, Q = quad word

12 Register handling

There is a flag to set the register handling by the Brackets compiler.

12.1 reset-reg

With the “reset-reg” flag the current variables which are in the registers will be unset in the registers.

This is needed to load variables into the registers again to access them. I use this in deep nested code with alternate branches that can be happen. You can use the flag like this: “(reset-reg)”.

13 Latest changes

There are some changes in the compiler (L1VM 3.1.4 and later) how the math expressions can be written. The new `:=` operator can be used on assign expressions.

13.1 Normal math expressions

Now the `:=` operator can be used to assign math expressions to a variable:

```
(a + (b - c) x :=)
(x ret :=)
```

In this example the `:=` assign operator is on the right side.

13.2 RPN

Math expressions can also be written in RPN (reverse polish notation).

```
(x y * z :=)
(a b + c d * + x :=)
```

In infix this would be:

```
(x * y z :=)
((a + b) + (c * d) x :=)
```

13.3 Arrays

Arrays can be assigned with the `:=` operator as assign too:

```
// assign variable xd to array element zero
```

```
(xd array [ zero ] :=)

// assign array element zero to variable zd
(array [ zero ] zd :=)
```

13.4 Multiline arrays

Array definitions can be split up in multiple lines. Here is an example from “prog/array-test-4.llcom”:

```
// set array data
(set int64 8 array 10 5 /)
@, 23, 42,
@, 64, 1234567890, 4, 3, ;
```

So the elements are: 10, 5, 23, 42, 64, 1234567890, 4 and 3.

13.5 Compatibility

All math and assign expressions can also be written in the older style. But it’s deprecated!

14 Build L1VM as a shared library

This is a new feature in the latest L1VM version.

14.1 Configure the build and build

Change the setting in “include/settings.h”:

```
#define L1VM_EMBEDDED 1
```

And save the settings file. Now go into the “vm” directory. There are zerobuild build scripts with “embedded” in their name. If you are using Linux then you need to type this in the shell:

```
CC=clang CXX=clang++ zerobuild zerobuild-nojit-embedded.txt force
```

Now clang should compile the “libl1vm-embedded.so” library. Note if you are using Windows then you have to use: “zerobuild-nojit-embedded-win.txt”. The macOS build file ends with “-macos”.

Finally you can copy the library into the “~/l1vm/bin” directory in your home directory.

So the L1VM library can be found by the programs linking it.

14.2 A C example using the L1VM library

Here is a full example. It shows how to use the L1VM library functions from a C program.

It runs the “Hello world!” program and shows how to access the global data.

To build the demo program you need to get into the directory “vm/test-embddedded/”.

You have to call zerobuild like this in the shell:

```
CC=clang CXX=clang++ zerobuild force
```

This builds the “test-l1vm” program. Here is the full C source code:

```
// test-l1vm.c
//
// This is a demo program of how to use the L1VM as a shared library program

#include <stdio.h>
#include <stdlib.h>
```

```

#include <string.h>

#include "l1vm-embedded.h"

int main (void) {
    long long int data_size, i, pwrap = 0;
    unsigned char *data_ptr = NULL;
    int ret;
    unsigned char hexstr[3];

    // this shows how to set command line arguments for the embedded L1VM
    // setting the stack size to 10000 as an example.
    int ac = 3;
    char *av[3] = {"l1vm", "-S", "10000"};

    printf("starting L1VM hello world program...\n");

    ret = l1vm_run_program ("hello", ac, av);
    if (ret != 0)
    {
        printf ("error running program! exit!\n");
        l1vm_cleanup ();
        exit (1);
    }

    data_size = l1vm_get_global_data_size ();
    data_ptr = l1vm_get_global_data ();

    printf ("\n\nmemdump global data:\n");
    for (i = 0; i < data_size; i++)
    {
        sprintf ((char *) hexstr, "%X", data_ptr[i]);

        if (strlen ((const char *) hexstr) < 3)
        {
            printf ("0");
        }

        printf ("%s", hexstr);

        pwrap++;
        if (pwrap == 16)
        {
            printf ("\n");
            pwrap = 0;
        }
    }
    printf ("\n\n");

    // call the L1VM memory cleanup function:
    l1vm_cleanup ();
    exit (0);
}

```

This shows that you can pass the same command line arguments like on the normal standalone L1VM. You also can access the global data array. You can get the memory addresses out of the assembly code of your program. So you can see where a variable starts and ends. Make sure to call the “l1vm_cleanup” function at program end. This frees all memory allocated by the VM.

15 The preprocessor

The preprocessor can set macros as functions in “include/intr-func.llh”.

15.1 The definitions

Variable types can be set by setting them as the first char of a variable name:

“i” is an int64 number, “d” is a double number, “s” is a string and “o” an operator like “=”, “!=” and “||”.

```
#func print_s (sX) :(6 sX 0 0 intr0)
#func print_n (NONE) :(7 0 0 0 intr0)
```

This is just an example of “intr-func.h”. Note: if the type is not set then the check will always be true! The “NONE” means that no arguments are needed.

This is for backwards compatibility.

The old way of using macros was this:

```
print_s (hello_str)
print_n
```

In the new way both are called like functions:

```
(hello_str :print_s !)
(:print_n !)
```

15.2 String constants

Now with L1VM 3.2.0 it is possible to declare string constants. The L1VM breaks with an exception, if a constant string would be changed.

The declaration looks like this:

```
(set const-string s hellostr "Hello")
```

The preprocessor inserts this after the declaration:

```
(34 hellostraddr 0 0 intr0)
```

This sets the string “hellostr” as immutable on runtime. Every change results in a runtime exception!

15.3 Special chars

In a string special chars can be set by:

```
@@@q  “
@@@c  ,
@@@s  /
```

Here is an example:

```
(set string s blogurl_str (“https:@@s@@@smidnight-coding.de@@@sblog@@@sl1vm@@@s”))
```

This sets the string: “https://midnight-coding.de/blog/l1vm/”.

16 The stack/function calls

In the later versions of L1VM there are new symbols for the stack opcodes.

16.1 The new stack.llh include

```
>> stack.llh
>> use new stack operators, but set the normal commands
>> to avoid compatibility trouble with older compilers
#define b>| stpushb
#define i>| stpushi
#define d>| stpushd
#define |>b stpopb
#define |>i stpopi
#define |>d stpopd
#define >| stpush
#define |> stpop
```

The current compiler has this already build in. This are the short writing stack definitions.

They can be used like this:

```
(foo >|)
```

This pushes the variable “foo” to the stack.

```
(foo |>)
```

This pops the variable “foo” from the stack. The other symbols work the same way:

```
(bar b>|)
```

This pushes the byte variable “bar” to the stack.

To pop it this is used:

```
(bar |>b)
```

16.2 Clear the stack

The stack can be cleared with:

```
(:stack_clear !)
```

This clears the stack and the stack pointer is at the top, with no variable on it.

It can be useful to call this function if a new thread function was started. After the variables from the caller are taken from the stack! This is a feature of the current version of LIVM on GitHub.

16.3 Function calls

A function is called with it’s arguments:

```
(x y :add_int !)
```

This puts the “x” and “y” variable on the stack and calls the function “add_int”. In the function “add_int” you have to pop the arguments from the stack in reverse order! The stack is “LIFO”, last in first out. The stack knows which type a variable has. So if you get a runtime error on the stack, this means that something went wrong. For example the stack is empty on a “pop” instruction. Or the variable type you want to “pop” don’t match.

This are the stack push opcodes:

stpushb (byte), stpushi (integer), stpushd (double float), stpush (any type)

And the pop opcodes:

stpopb (byte), stpopi (integer), stpopd (double float), stpop (any type)

17 If statements

In Brackets there are “if”, “if+”, “else” and “endif” statements.

To use “else” the “if+” must be set at the beginning! So the compiler knows that after “if+” there must be an “else” following. Here is an example:

```
// if-4.llcom
#include <intr.llh>
(main func)
  (set int64 1 zero 0)
  (set int64 1 one 1)
  (set int64 1 x 6)
  (set int64 1 y 10)
  (set int64 1 z 5)
  (set int64 1 twen 20)
  (set int64 1 f 0)
  (set string s less_str "x < 10")
  (set string s more_str "x ==> 10")
  (set string s more_less_20_str "x <= 20")
  (set string s five_less_str "x <= 5")
  (set string s five_more_str "x > 5")

  // check if x is less or more ten
  (((x y <) f =) f if+)
    (less_str :print_s !)
    (((x z <=) f =) f if+)
      (five_less_str :print_s !)
    (else)
      (five_more_str :print_s !)
    (endif)
  (else)
    (more_str :print_s !)
    (((x twen <=) f =) f if)
      (more_less_20_str :print_s !)
    (endif)
  (endif)
  (:print_n !)
  (zero :exit !)
(funcend)
```

Here is the output:

```
$ llvm prog/if-4 -q
x < 10 x > 5
```

Here is another example, with the new “:=” assign:

```
// math-test-3.llcom
// Brackets - Hello world!
//
#include <intr-func.llh>
(main func)
  (set int64 1 zero 0)
  (set int64 1 x 100)
  (set int64 1 f 0)
  (set int64 1 range_max 100)
  (set int64 1 range_min 10)
  (set int64 1 max 42)
  (set int64 1 y 43)
  (set string s range_okstr "range ok!")
  (set string s range_outstr "range outside!")
```

```

(((x >= range_min) && (x <= range_max) && (y < max)) f :=)
(f if+)
    (range_okstr :print_s !)
    (else)
        (range_outstr :print_s !)
    (endif)
    (:print_n !)
    (zero :exit !)
(funcend)

```

The output:

```

t$ llvm math-test-3 -q
range outside!

```

18 Switch

There is a “switch” statement in Brackets. It works like the “if” statement. If a variable has a value which matches, then the code inside is executed.

Here is an example:

```

// switch.llcom
// Brackets - Hello world! switch
//
#include <intr-func.llh>
(main func)
    (set int64 1 zero 0)
    (set int64 1 x 23)
    (set int64 1 y 42)
    (set string s 23_str "y = 23")
    (set string s 42_str "y = 42")
    (set const-int64 1 23_const 23)
    (set const-int64 1 42_const 42)
    (set int64 1 a 0)
    (set string s hello_str "Hello world!")

    (hello_str :print_s !)
    (:print_n !)
    (switch)
        (y 23_const ?)
            (23_str :print_s !)
            (:print_n !)
        (break)
        (y 42_const ?)
            (42_str :print_s !)
            (:print_n !)
        (break)
    (switchend)
    (zero :exit !)
(funcend)

```

The “break” is like an “endif” in an “if” statement. So all variable comparisons in the switch are done and the matched code is executed.

Here is the output:

```

$ llvm prog/switch -q
Hello world!
y = 42

```

19 For loops

Now In Brackets 3.2.0 “for” loops can be defined without the “(for-loop)” start line.

19.1 The new way

Here is a full example:

```
// hello-for-3 - Brackets - Hello world! do loop
//
#include <intr-func.llh>
(main func)
    (set int64 1 zero 0)
    (set int64 1 one 1)
    (set int64 1 loop 0)
    (set int64 1 maxloop 10)
    (set int64 1 f 0)
    // set string length automatic, by setting "s"
    (set string s hello "Hello world!")
    // print string
    // for
    (zero loop :=)
    (((loop maxloop <) f :=) f for)
    (hello :print _s !)
    // print newline
    (:print _n !)
    ((loop one +) loop :=)
    (next)
    (zero :exit !)
(funcend)
```

20 Do while loops

A do while loop is executed at least once. Here is an example:

```
// hello-while - Brackets - Hello world! do loop
//
#include <intr.llh>
(main func)
    (set int64 1 zero 0)
    (set int64 1 one 1)
    (set int64 1 loop 0)
    (set int64 1 maxloop 10)
    (set int64 1 f 0)
    // set string length automatic, by setting "s"
    (set string s hello "Hello world!")
    // print string
    // while
    (zero loop =)
    (do)
        (hello :print _s !)
        // print newline
        (:print _n !)
        ((loop one +) loop =)
    (((loop maxloop <) f =) f while)
    (zero :exit !)
(funcend)
```

21 Operators

In Brackets there are logical operators: “|” (or) and “&&” (and). This is like in C. The logical operators can be used in math expressions and on the “if” statement.

21.1 And or and shifting

```
// math-test-2.llcom - Brackets - new math expression test
#include <intr-func.llh>
(main func)
  (set int64 1 zero 0)
  (set int64 1 xd 23)
  (set int64 1 yd 42)
  (set int64 1 zd 0)
  (set int64 1 zerod 0)
  (set int64 1 i 0)
  (set int64 1 f 0)
  (set int64 1 a 2)
  (set int64 1 b 0)
  (set int64 1 shift 5)
  (set string s messagestr "xd < yd and xd > 0")

  // new math expression:
  ((xd < yd) && (xd > zerod) i :=)
  (i :print_i !)
  (:print_n !)
  (i if)
    (messagestr :print_s !)
    (:print_n !)
  (endif)
  (((xd yd <)(xd zerod >) &&) f =) f if)
    (messagestr :print_s !)
    (:print_n !)
  (endif)
  // shift
  {b = (a <| shift)}
  (b :print_i !)
  (:print_n !)

  {b = (b >| shift)}
  (b :print_i !)
  (:print_n !)

  (zero :exit !)
(funcend)

$ llvm math-test-2 -q
1
xd < yd and xd > 0
xd < yd and xd > 0
64
2
```

21.2 Other logical operators

There are more logical operators: “&” is a binary and, “|” is a binary or, “^” is a binary xor and “%” is a modulo operator.

21.3 Here is an example

```
(x & y z :=)
```

This calculates the binary “and” of “x” and “y”, stored in the “z” variable.

21.4 Math operators

There are the normal math operators like: add “+”, subtract “-“, multiply “*” and division “/“.

And there are the old math operators for the floating point numbers: “+d”, “-d”, “*d” and “/d“.

Here is an example with double float numbers:

```
((xd yd +d) zd =)
```

In the modern syntax this can be written as:

```
(xd + yd zd :=)
```

22 Shell args

With the function “shell_args” the number of CLI args can be read. With “get_shell_arg” the string on an index can be read.

Here is an example:

```
// shell-args.llcom - show shell arguments
#include <intr-func.llh>
(main func)
    (set int64 1 zero 0)
    (set int64 1 one 1)
    (set int64 1 args 0)
    (set string 256 shell_arg "")
    (set int64 1 i 0)
    (set int64 1 f 0)
    (set string s argstr " shell arguments: ")

    // get number of shell arguments:
    (args :shell_args !)
    (args :print_i !)
    (argstr :print_s !)
    (:print_n !)
    (:print_n !)

    (((args zero ==) f =) f if)
    // no arguments, exit!
    (one :exit !)
    (endif)
    (zero i =)
    (:loop)
    // get shell argument in variable "shell_arg"
    (i shell_arg :get_shell_arg !)
    (shell_arg :print_s !)
    (:print_n !)

    ((i one +) i =)
    (((i args <) f =) f if)
    (:loop jmp)
    (endif)
    (zero :exit !)
(funcend)
```

The output:

```
llvm prog/shell-args -q -args "Hello world!"
1 shell arguments:

Hello world!
```

23 CLI input

There are three input functions: “input_i” (int64), “input_d” (double) and “input_s” (string). Here is an example which reads a string:

```
// hello-input.llcom
#include <intr-func.llh>
(main func)
    (set const-int64 1 zero 0)
    (set string s inputstr "text input? ")
    (set int64 1 strlen 10)
    (set string 10 str "")

    (inputstr :print_s !)
    (strlen str :input_s !)

    (str :print_s !)
    (:print_n !)

    (zero :exit !)
(funcend)
```

24 Objects

In Brackets objects can be defined. The shared variables are defined at the beginning.

The object functions are defined like this:

```
(Pshow_data->person func)
```

Here the object name is “person”. The “P” in “Pshow_data” marks the function as “private”.

It can only be called from the other functions inside the object.

Here is a full example:

```
// people-object-oop.llcom
// use . variable names to build something like a pseudo data structure like in other
// programming languages
//
// NEW: using:
// #var ~ main to be more safe!
// using object for data and functions
#include <intr-func.llh>
#include <vars.llh>
(main func)
    #var ~ main

    (set int64 1 zero 0)
    (set int64 1 one 1)
    // person data with person. names
    (set string 256 person.first_name~ "Jane")
    (set string 256 person.last_name~ "Doe")
    (set string 256 person.gender~ "female")
```

```

(set int64 1 person.age~ 30)
(set double 1 person.height~ 1.80)
(set string 256 person.eye_color~ "brown")
(set string s meminitstr~ "ERROR can't init memory!")
(set string s memerrstr~ "ERROR can't allocate memory!")
(set string s saveerrstr~ "ERROR can't save to memory object!")
(set int64 1 memaddr~ 0)
(set int64 1 memind~ 0)
(set int64 1 memsize~ 1)
(set int64 1 variables~ 6)
(set int64 1 ret~ 0)
(set int64 1 f~ 0)
// include memory object variable types
#include <mem-obj-lib-types.llh>
(zero one :mem_obj_init !)
(ret~ stpopi)
(((ret~ zero !=) f~ =) f~ if)
  // ERROR can't allocate memory
  (meminitstr~ :print_s !)
  (:print_n !)
  (one :exit !)
(endif)
// allocate memory for the 6 variables
(variables~ memsize~ :alloc_obj_memobj !)
(memaddr~ stpopi)
(loadreg)
(((memaddr~ zero <) f~ =) f~ if)
  // ERROR can't allocate memory
  (memerrstr~ :print_s !)
  (:print_n !)
  (one :exit !)
(endif)

// store person data in mem object using memory variable types: s_type, i_type and d_type
// going from right to left with variable types first before variable
  (person.eye_color~ s_type person.height~ d_type person.age~ i_type person.gender~
s_type person.last_name~ s_type person.first_name~ s_type variables~ zero
memaddr~ :save_obj_memobj !)
  (ret~ stpopi)
  (((ret~ zero !=) f~ =) f~ if)
  // ERROR can't allocate memory
  (saveerrstr~ :print_s !)
  (:print_n !)
  (one :exit !)
(endif)
(memaddr~ :list_data->person !)
// free memory
(zero :free_obj_memobj !)
(:free_memobj !)
(zero :free_mod !)
(zero :exit !)
(funcend)

// set variable ending check on: only local variables ending with "person" or globals ending
with "main" allowed!
(variable-local-on)

// get person data from mem object
(person object)
#var ~ person

```



```

(set int64 1 f~)
(set int64 1 zero~ 0)
(set int64 1 variables~ 6)
(set int64 1 memaddr~ 0)
(set int64 1 ret~ 0)
(set string s height_format~ "%.2lf")
(set string 256 first_name~ "")
(set string 256 last_name~ "")
(set string 256 gender~ "")
(set int64 1 age~ 0)
(set double 1 height~ 0.0)
(set string 256 eye_color~ "")
(set string s memobj_err_str~ "ERROR reading memobject array!")
// data pointer, make only changeable by "pointer" function to avoid memory errors
(set const-int64 1 Pfirst_name~ 0)
(set const-int64 1 Plast_name~ 0)
(set const-int64 1 Pgender~ 0)
(set const-int64 1 Page~ 0)
(set const-int64 1 Pheight~ 0)
(set const-int64 1 Peyeye_color~ 0)

(Pshow_data->person func)
  #var ~ person

  (Peye_color~ Pheight~ Page~ Pgender~ Plast_name~ Pfirst_name~ variables~ zero~
memaddr~ :load_obj_array_memobj !)
  (ret~ stpopi)
  (((ret~ zero~ !=) f~ =) f~ if+)
  (memobj_err_str~ :print_s !)
  (:print_n !)
  (else)
    (first_name~ :print_s !)
    (:print_n !)
    (last_name~ :print_s !)
    (:print_n !)
    (gender~ :print_s !)
    (:print_n !)
    (age~ :print_i !)
    (:print_n !)
    (height_format~ height~ :printf_d !)
    (:print_n !)
    (eye_color~ :print_s !)
    (:print_n !)
  (endif)
(funcend)

(variable-local-off)

(list_data->person func)
  #var ~ person

  // get mem address of person data
  (memaddr~ stpopi)

  // set pointers, use new check_pointer interrupt for pointer type checks
  (first_name~ Pfirst_name~ pointer)
  (Pfirst_name~ byte_ptr :check_pointer !)

  (last_name~ Plast_name~ pointer)

```

```

(Plast_name~ byte_ptr :check_pointer !)

(gender~ Pgender~ pointer)
(Pgender~ byte_ptr :check_pointer !)

(age~ Page~ pointer)
(Page~ int64_ptr :check_pointer !)

(height~ Pheight~ pointer)
(Pheight~ double_ptr :check_pointer !)

(eye_color~ Peyecolor~ pointer)
(Peyecolor~ byte_ptr :check_pointer !)

(:Pshow_data->person !)
(funcend)
(objectend)

#include <mem-obj-lib.l1h>

```

25 Pointer type checking

Here I want to show how pointers can be used to go different paths in a program.

The print function checks the pointer type and then uses the right "print" function for that type.

In other languages like C++ you would do function overloading with templates. This is not needed in Brackets, as we can check the pointer type.

25.1

Here is the example:

```

// hello-print.l1com
// This shows how to use the pointer_type interrupt to get the pointer array variable type.
//
#include <intr-func.l1h>
#include <vars.l1h>
#include <misc-macros.l1h>
(main func)
  (set const-int64 1 zero 0)
  (set int64 1 x 23)
  (set int64 1 y 42)
  (set int64 1 a 0)
  (set string 13 hello "Hello world!")
  (set double 1 d 123.456)
  (set int64 1 Pvar 0)

  // get string pointer
  (hello Pvar pointer)
  (Pvar :print_var !)
  (:print_n !)

  ((x y *) a :=)
  // get int64 pointer
  (a Pvar pointer)
  (Pvar :print_var !)
  (:print_n !)

  // get double pointer
  (d Pvar pointer)

```

```

(Pvar :print _var !)
(:print _n !)

exit (zero)
(funcend)

(print _var func)
// Print variable the pointer points at.
#var ~ print _var
(set const-int64 1 zero~ 0)
(set int64 1 Pvar~ 0)
(set int64 1 vartype~ 0)
(set int64 1 f~ 0)
(set int64 1 int _var~)
(set double 1 double _var~)
(set string s stringvar~ "")
(Pvar~ stpop)

// get pointer type
(Pvar~ vartype~ :pointer _type !)
// pull vartype~ to variable
(vartype~ :pull _int64 _var !)

// Check pointer type and print the variable.
(switch)
  (vartype~ byte_ptr ?)
    (Pvar~ stringvar~addr :=)
    (stringvar~ :print _s !)
    (break)

  (vartype~ int64_ptr ?)
    (Pvar~ [ zero~ ] int _var~ :=)
    (int _var~ :print _i !)
    (break)

  (vartype~ double_ptr ?)
    (Pvar~ [ zero~ ] double _var~ :=)
    (double _var~ :print _d !)
    (break)
(switchend)
(funcend)

```

Here is the output:

```

$ llvm prog/hello-print -q
Hello world!
966
123.4560000000

```

26 Testing

There is now a script for testing: “test-compare.sh” in “prog/unit-test”.

26.1 Preparation

Here I will show how to run a test program checking the output of a program.

To start copy “test-compare.sh” to the “~/llvm/bin” directory.

Here is the function we use: “multiply.llh” in a header file:

```
(multiply func)
  #var ~ multiply

  (set int64 1 x~ 0)
  (set int64 1 y~ 0)
  (set int64 1 ret~ 0)
  (y~ x~ stpop)
  {ret~ = (x~ * y~)}
  (ret~ stpush) (return)
(funcend)
```

This is “multiply.llcom” the main program:

```
// multiply.llcom
//
// (func args multiply int64 int64)
// (return args multiply int64)
//
#include <intr.llh>
(main func)
  (set int64 1 zero 0)
  (set int64 1 x 5)
  (set int64 1 y 3)
  (set int64 1 ret 0)
  (x y :multiply !) (ret stpop)
  (ret :print _i !)
  (:print _n !)
  (zero :exit !)
(funcend)
#include <multiply.llh>
```

It includes the header file.

Here in the comments the argument types for the function "multiply" are set.

And the return variable types. They are both checked by the linter.

Here is the test program "multiply-test.llcom":

```
// multiply-test.llcom
//
// (func args multiply int64 int64)
// (return args multiply int64)
//
#include <intr-func.llh>
(main func)
  (set int64 1 zero 0)
  (set int64 1 one 1)
  (set int64 1 x1 5)
  (set int64 1 y1 3)

  (set int64 1 x2 -5)
  (set int64 1 y2 3)

  (set int64 1 ret 0)
  (set int64 1 f 0)
  (set int64 1 retcode 0)
  // run tests

  // test 1
  (x1 y1 :multiply !) (ret stpop)
```

```

    (ret :print _i !)
    (:print _n !)

    // test 2
    (x2 y2 :multiply !) (ret stpop)

    (ret :print _i !)
    (:print _n !)

    (retcode :exit !)
(funcend)
#include <multiply.llh>

```

Now we are ready to build the “multiply.llcom” program:

```

$ llvm-build-lint.sh multiply
building: multiply
llvm-linter 3.3.3 , no errors found!
code lines compiled: 106
[✓] out compiled
assembler args: ' '
assembling file: 'out'
codesize: 277 bytes
datasize: 456 bytes
filesize: 804 bytes
[✓] out assembled

build in 0.0045 seconds
copying files...

```

So everything is fine! Now lets build the test program "multiply-test.llcom":

```

$ llvm-build-lint.sh multiply-test
building: multiply-test
llvm-linter 3.3.3 , no errors found!
code lines compiled: 126
[✓] out compiled
assembler args: ' '
assembling file: 'out'
codesize: 382 bytes
datasize: 776 bytes
filesize: 1274 , 1.244 KB
[✓] out assembled

build in 0.0047 seconds
copying files...

```

It's all ok!

Now we need to set the correct test output in a file "multiply-test.txt":

```

15
-15

```

26.2 Run the test

We now are ready to run the test script "multiply-test.sh":

```

#!/bin/bash
llvm multiply-test -q >multiply-test-out.txt

```

```
test-compare.sh multiply-test.txt multiply-test-out.txt
```

The output of the script is:

```
$ ./multiply-test.sh
test passed, OK!
```

The test passed, everything was ok. The tests output was the one as set in “multiply-test.txt”.

You can write tests like this to check a program output matches the test set in a file.

All the example files are in the L1VM GitHub repository. You can use them as a template for your own programs and tests.

27 Hot code reload

In Brackets there is a function to reload a new code at runtime.

27.1 Intro

You can call a special interrupt with a bytecode file name.

It then loads this bytecode. And in the VM main program a flag is set to restart the VM.

The new bytecode begins running from the beginning. All the data is not changed and stays the same.

To make this work you can't change data addresses in the new bytecode file!

27.2 Example code

Here is an example from "prog/reload":

```
// hello-reload.llcom
// Brackets - bytecode hot reload demo
// Only the new bytecode is loaded.
// The program starts from the beginning of the new code.
#include <intr-func.llh>
(main func)
  (set const-int64 1 zero 0)
  (set const-int64 1 one 1)
  (set int64 1 x 23)
  (set int64 1 y 42)
  (set int64 1 a 0)
  (set const-string s hello_str "Hello world!")
  (set const-string s bytecode_str "./hello-reload-2")
  (set int64 1 reload 0)
  (set int64 1 f 0)
  // print string
  (hello_str :print_s !)
  (:print_n !)
  (x * y a :=)
  (a :print_i !)
  (:print_n !)
  (((reload zero ==) f :=) f if)
  (one reload :=)

  // do reload
  (bytecode_str :load_bytecode !)
  (endif)
  (zero :exit !)
(funcend)
```

This program is hot reloaded:

```
// hello-reload-2.llcom
#include <intr-func.llh>
(main func)
  (set const-int64 1 zero 0)
  (set const-int64 1 one 1)
  (set int64 1 x 23)
  (set int64 1 y 42)
  (set int64 1 a 0)
  (set const-string s hello_str "Hello world!")
  (set const-string s bytecode_str "./hello-reload-2")
  (set int64 1 reload 0)
  (set int64 1 f 0)
  // print string
  (hello_str :print_s !)
  (:print_n !)
  (x + y a :=)
  (a :print_i !)
  (:print_n !)
  (zero :exit !)
(funcend)
```

The second one does an addition instead of a multiplication!

Here is the output of the two programs:

```
$ llvm hello-reload -q
Hello world!
966
Hello world!
65
```

Don't forget that only the code can be changed. The variable addresses must be the same!

So you can't add new variables in the new code!

28 Unsafe code blocks

This is a new feature in the latest version of Brackets.

In an unsafe code block the variable bounds checking can be turned off:

```
(unsafe)
(:memory_bounds_off !)

// code block

(:memory_bounds_on !)
(unsafe-end)
```

A full code example is: in “prog/little-vm.llcom”. With the memory bounds checking off some programs run faster. But the safety checks are then done no more inside the code block!

29 Contracts

In Brackets contracts can be used to set conditions. There are “preconditions” and “postconditions” for a function. This code blocks must contain valid code and can not be empty!

The contracts are turned on and off via a flag:

```
(contracts-on)
(contracts-off)
```

29.1 Here is an example

```
// math-inc.llcom
//
// contracts: precondition/postcondition demo
//
// (func args incP int64)
// (return args incP int64 int64)
(linter)

#include <intr-func.llh>

(main func)
  (set const-int64 1 zero 0)
  (set const-int64 1 one 1)
  (set int64 1 x 42)
  (set int64 1 incx 0)
  (set int64 1 f 0)
  (set int64 1 ret 0)
  (set const-string s error_inc_str "inc: error!")

  (x :incP !) (ret incx stpop)
  (((ret zero !=) f :=) f if)
    (error_inc_str :print_s !)
    (:print_n !)
    (one :exit !)
  (endif)

  (incx :print_i !)
  (:print_n !)

  (zero :exit !)
(funcend)

(contracts-on)
(variable-local-only-on)

(incP func)
  #var ~ incP
  (set int64 1 num~ 0)
  (set int64 1 ret~ 0)
  (set int64 1 check~ 0)
  (set const-int64 1 zero~ 0)
  (set const-int64 1 one~ 1)
  (set int64 1 f~ 0)
  (set int64 1 int64-max~ 9223372036854775807)
  (set const-string s overflow_str~ "inc: error number overflow!")
  (set const-string s error_int_str~ "inc: error internal!")

  (num~ stpop)

  (precondition)
  (((num~ int64-max~ ==) f~ :=) f~ if)
    (overflow_str~ :print_s !)
    (:print_n !)
    (zero~ one~ stpush) (return)
```



```

(endif)
(precondition-end)

(num~ + one~ ret~ :=)

(postcondition)
(ret~ - one~ check~ :=)
(((check~ num~ !=) f~ :=) f~ if)
  (error_int_str~ :print_s !)
  (:print_n !)
  (zero~ one~ stpush) (return)
(endif)
(postcondition-end)

(ret~ zero~ stpush) (return)
(funcend)

```

The output:

```

$ llvm math-inc -q
43

```

II - The modules

In Brackets modules can be load and their functions be called. A module has two parts: the C library code with the API which is called from the VM. And the include file for the module which handles the setting of the module functions. The functions of a module are called like the normal functions in Brackets. The function arguments are also copied on to the stack. And then taken from the C library function.

In this part of the manual I want to show some modules. I can't cover all here. You can also take a look at the examples in the "lib" directory. The examples I show here are also there.

30 The file module

The file module can read and write files. There is also a file-tools module with functions to copy, rename files and list the directories.

30.1 Normal functions example code

Here is an example code how to handle errors while read and write into a file:

```
// file-lib.llcom - file library module
// test program
#include <intr-func.llh>
(main func)
    #include <file-def.llh>
    (set int64 1 zero 0)
    (set int64 1 one 1)
    (set int64 1 handle 0)
    (set int64 1 num 12345678Q)
    (set int64 1 read 0)
    (set string s filename "test-file")
    (set string s hellostr "hello world!")
    (set string 30 readstr "")
    (set int64 1 readlen 29)
    (set byte 1 newline 10)
    (set byte 1 mode_write 2)
    (set byte 1 mode_read 1)
    (set byte 1 mode_readwrite 3)
    (set int64 1 error 0)
    (set int64 1 ret 0)
    (set int64 1 f 0)
    (set string s error_init_files "error: can't allocate file handles!")
    (set string s error_open_file "error: can't open file test-file!")
    (set string s write_error "error writing file!")
    (set string s read_error "error reading file!")
    //
    // set numbers of file handles to one: 1
    //
    (zero one :file_init call)
    (ret stpopi)
    (((ret zero !=) f =) f if)
        (error_init_files :print_s !)
        (:print_n !)
        (:file_free_mem !)
        (zero :free_mod !)
        (one :exit !)
    (endif)
```

```

// write data -----
(mode_write filenameaddr :file_open !)
//
// get file handle from file_open call
//
(handle stpopi)
(((handle zero <) f =) f if)
  (error_open_file :print_s !)
  (:print_n !)
  (:file_free_mem !)
  (zero :free_mod !)
  (one :exit !)
(endif)

(handle num :file_put_int64 !)
(error stpopi)
(((error zero !=) f =) f if)
  (write_error :print_s !)
  (:print_n !)
  (handle :file_close !)
  (:file_free_mem !)
  (zero :free_mod !)
  (one :exit !)
(endif)

(handle hellostraddr :file_put_string !)
(error stpopi)
(((error zero !=) f =) f if)
  (write_error :print_s !)
  (:print_n !)
  (handle :file_close !)
  (:file_free_mem !)
  (zero :free_mod !)
  (one :exit !)
(endif)
(handle :file_close call)

// read data -----
(mode_read filenameaddr :file_open call)
//
// get file handle from file_open call
//
(handle stpopi)
(((handle zero <) f =) f if)
  (error_open_file :print_s !)
  (:print_n !)
  (:file_free_mem !)
  (zero :free_mod !)
  (one :exit !)
(endif)

(handle :file_get_int64 !)
(error stpopi)
(read stpopi)
(((error zero !=) f =) f if)
  (read_error :print_s !)
  (:print_n !)
  (handle :file_close !)
  (:file_free_mem !)
  (zero :free_mod !)

```

```

    (one :exit !)
  (endif)

  (handle readstraddr readlen :file_get_string call)
  (error stpopi)
  (((error zero !=) f =) f if)
    (read_error :print_s !)
    (:print_n !)
    (handle :file_close !)
    (:file_free_mem !)
    (zero :free_mod !)
    (one :exit !)
  (endif)

  (readstr :print_s !)
  (:print_n !)
  (read :print_i !)
  (:print_n !)

  (handle :file_close !)
  (:file_free_mem !)
  (zero :free_mod !)
  (zero :exit !)
(funcend)
#include <file-lib.llh>

```

Here is the output:

```

$ llvm lib/file-lib -q
hello world!
12345678

```

30.2 File tools example

Here is an example program which renames a file:

```

// file-tools.llcom - file library module
#include <intr.llh>
(main func)
  #include <file-def.llh>
  (set int64 1 zero 0)
  (set int64 1 one 1)
  (set string s oldfilenamestr "foobar.txt")
  (set string s newfilenamestr "hello.txt")
  (set string s error_renamestr "error: can't rename file 'foobar.txt' to 'hello.txt'!")
  (set int64 1 ret 0)
  (set int64 1 f 0)
  (zero :file_tools_init !)

  // rename file
  (oldfilenamestr newfilenamestr :file_tools_rename !)
  (ret stpop)
  (((ret zero ==) f =) f if)
    (error_renamestr :print_s !)
    (:print_n !)
  (endif)
  (zero :free_mod !)
  (zero :exit !)
(funcend)

#include <file-tools-lib.llh>

```

31 The string module

In the string module there are functions like in the C string library. Strings can be concatenated. And the length can be get. There are also functions to get the left or right part of a string. And also to get a string part from inside a string. There are also functions to convert numbers into strings.

It is possible to parse JSON strings.

31.1 The string library example

```
// string.llcom
// now with string array demo
#include <intr-func.llh>
#include <vars.llh>
(main func)
  (set int64 1 zero 0)
  (set int64 1 one 1)
  (set string 14 hellostr "Hello")
  (set string s worldstr " world!")
  (set string 15 hellostr2 "")
  (set string 15 worldstr2 "")
  (set string 2 dotstr ",")
  (set int64 1 slen 0)
  (set string 30 bufstr "")
  (set int64 1 buflen 29)
  (set int64 1 bytenum 42)
  (set int64 1 hello_len 5)
  (set int64 1 world_len 6)
  (set int64 1 mid_pos 6)
  (set int64 1 dot_pos 5)
  //
  // set array with 30 chars
  // and 15 chars length for array strings
  (set string 30 arraystr "")
  (set int64 1 array_size 30)
  (set int64 1 string_len 15)
  (set string s regexstr "[A-Z]")
  (set string s regexsrcstr "Abcdefgh")
  (set string s regex_foundstr "regex: pattern found!")
  (set string s regex_notfoundstr "regex: pattern not found!")
  (set string s json_infostr "JSON get values:")
  // (set string s jsonstr "@@qtest@@q: @@qkey test@@q")
  (set string s jsonstr "@@qtest@@q: 12345.6789")
  (set string 256 keystr "")
  (set string 256 valustr "")
  (set int64 1 maxlen 255)
  (set int64 1 ret 0)
  (set int64 1 f 0)
  (set string s rep_in_str "Hello 123!")
  (set string 4097 rep_out_str "")
  (set string s search_rep_str "123")
  (set string s rep_str "world")
  (set string s sortastr "xyz")
  (set string s sortbstr "abcde")
  (set string s searchstr "cde")
  (set string 15 sortedastr "")
  (set string 15 sortedbstr "")
  (set int64 1 pos -1)
  (set int64 1 index -1)

  (set string s verify_charstr "range: 0-9 range: A-F-+/{&*")
```

```

(set string s verifystr "01234567890FEA+")
(set string s verify_okstr "string verify ok!")
(set string s verify_failstr "string verify failed!")

(zero :string_init !)
// copy hellostr to array
(hellostraddr arraystraddr zero string_len array_size :string_to_array !)
// copy worldstr to array
(worldstraddr arraystraddr one string_len array_size :string_to_array !)
// get strings from array
(arraystraddr hellostr2addr zero string_len array_size :array_to_string !)
(arraystraddr worldstr2addr one string_len array_size :array_to_string !)
(hellostr2 :print_s !)
(:print_n !)
(worldstr2 :print_s !)
(:print_n !)
(:print_n !)
(hellostraddr worldstraddr :string_cat !)
(hellostr :print_s !)
(:print_n !)
(hellostraddr :string_len !)
(slen stpopi)
(slen :print_i !)
(:print_n !)
(hellostraddr hellostr2addr hello_len :string_left !)
(hellostr2 :print_s !)
(:print_n !)
(hellostraddr worldstr2addr world_len :string_right !)
(worldstr2 :print_s !)
(:print_n !)
(slen bufstr buflen :string_int64tostring !)
(bufstr :print_s !)
(:print_n !)
(bytenum bufstr buflen :string_bytetostring !)
(bufstr :print_s !)
(:print_n !)
// get char at position mid_pos in hellostr
(hellostraddr bufstr mid_pos :string_mid !)
(bufstr :print_s !)
(:print_n !)
(dotstraddr hellostraddr dot_pos :string_to_string !)
(hellostr :print_s !)
(:print_n !)
// check regex function:
(regexsrcstraddr regexstr :string_regex !)
(ret stpopi)
(((ret zero ==) f =) f if+)
  (regex_foundstr :print_s !)
  (:print_n !)
(else)
  (regex_notfoundstr :print_s !)
  (:print_n !)
(endif)

// JSON parse function
(json_infostr :print_s !)
(:print_n !)
(jsonstr :print_s !)
(:print_n !)
(jsonstr keystr valustr maxlen :string_parse_json !)

```

```

(ret stpop)
(ret :print_i !)
(:print_n !)
(keystr :print_s !)
(:print_n !)
(valustr :print_s !)
(:print_n !)
// replace string
(rep_in_str search_rep_str rep_str rep_out_str :string_replace !)
(rep_out_str :print_s !)
(:print_n !)
// copy hellostr to array
(sortastraddr arraystraddr zero string_len array_size :string_to_array !)
// copy worldstr to array
(sortbstraddr arraystraddr one string_len array_size :string_to_array !)
// sort string array
(arraystr zero string_len array_size sort_string_asc :string_array_sort !)
(ret stpop)
// print sorted array
(arraystraddr sortedastraddr zero string_len array_size :array_to_string !)
(arraystraddr sortedbstraddr one string_len array_size :array_to_string !)
(sortedastr :print_s !)
(:print_n !)
(sortedbstr :print_s !)
(:print_n !)
// search for string in array
(arraystraddr searchstraddr zero string_len array_size :string_array_search !)
(index pos stpop)
(index :print_i !)
(:print_n !)
(pos :print_i !)
(:print_n !)

(verifystr verify_charstr :string_verify !)
(ret stpop)
(((ret zero ==) f =) f if+)
    (verify_okstr :print_s !)
(else)
    (verify_failstr :print_s !)
(endif)
(:print_n !)

(zero :free_mod !)
(zero :exit !)
(funcend)
#include <string.h>
#include <string-tools.h>

```

The output:

```
$ llvm string -q
Hello
world!

Hello world!
12
Hello
world!
12
2a
w
Hello,world!
regex: pattern found!
JSON get values:
"test": 12345.6789
0
test
12345.6789
Hello world!
abcde
xyz
0
2
string verify ok!
```

32 The Unicode string module

With the Unicode string module Unicode strings can be printed. There are functions to set them by their Unicode char codes. Also the length of a Unicode string can be get.

32.1 The Unicode string example

```
// string-unicode.llcom
#include <intr-func.llh>
#include <vars.llh>
(main func)
  (set int64 1 zero 0)
  (set int64 1 one 1)
  (set int64 1 str_unicode_mod 0)
  (set string 5 unicode_str "")
  (set int64 1 unicode_copyright 169)
  (set string s unicode_len_str "unicode char length: ")
  (set string 5 partstr "")
  (set int64 1 ret 0)
  (set int64 1 strlen 0)
  (set int64 1 f 0)
  (set string s partstr_errstr "error: can't get unicode part string!")

  (str_unicode_mod :string_unicode_init !)
```



```

(unicode_copyright unicode_str :unicode_codepoint_to_string !)
(unicode_str :print_s !)
(:print_n !)

(unicode_str :unicode_string_to_codepoint !)
(ret stpopi)
(ret :print_i !)
(:print_n !)

// Get utf8 char length, multi byte chars count as one!
(unicode_str :unicode_strlen !)
(strlen stpopi)
(unicode_len_str :print_s !)
(strlen :print_i !)
(:print_n !)

// get unicode part string
(unicode_str zero partstr :unicode_partstr !)
(ret stpopi)
(((ret zero !=) f =) f if+)
    (partstr_errstr :print_s !)
    (:print_n !)
(else)
    (partstr :print_s !)
    (:print_n !)
(endif)

(str_unicode_mod :free_mod !)
(zero :exit !)
(funcend)
#include <string-unicode.llh>

```

Here is the output:

```

$ llvm string-unicode -q
©
169
unicode char length: 1
©

```

33 The math module

The math functions are in the “math-lib.llh” include file. There are the functions like in the C math library. There are functions to convert a string into a number. And there is an include file with math constants “math-const.llh”.

33.1 An example

```
// math-lib.llh com
#include <math-const.llh>
#include <intr-func.llh>
(main func)
    (set int64 1 zero 0)
    (set int64 1 randstart 2003)
    (set int64 1 random 0)
    (set int64 1 digits 3)
    (set int64 1 numstr_len 30)
    (set int64 1 not_num 0)
    (set int64 1 not_ret)
    (set double 1 angle 45.0)
    (set double 1 rad)
    (set double 1 angle_sin)
    (set string 30 numstr "")
    (set string s anglestr "angle sin: ")
    (set string s anglestr_2 " = ")
    (zero :math_init !)
    (randstart :math_randinit !)
    (:math_randint !)
    (random stpopi)
    // print random number
    (random :print_i !)
    (:print_n !)

    // print pi
    (m_pimath :print_d !)
    (:print_n !)

    // round pi to "digits" (3) digits and store number in string
    (m_pimath digits numstraddr numstr_len :double_rounded_string !)
    (numstr :print_s !)
    (:print_n !)

    // get "not" of not_num (0)
    (not_num :math_not !)
    (not_ret stpopi)
    (not_ret :print_i !)
    (:print_n !)

    // convert degree to rad
    (angle :math_degree_to_rad !)
    (rad stpopd)
    (rad :math_sindouble !)
    (rad stpopd)
    (anglestr :print_s !)
    (angle :print_d !)
    (anglestr_2 :print_s !)
    (rad :print_d !)
    (:print_n !)

    // close module
    (zero :free_mod !)
```

```

    (zero :exit !)
(funcend)

#include <math-lib.llh>

```

Here is the output:

```

$ llvm lib/math-lib -q
-5419689098295549502
3.1415926536
3.142
1
angle sin: 45.0000000000 = 0.7071067812

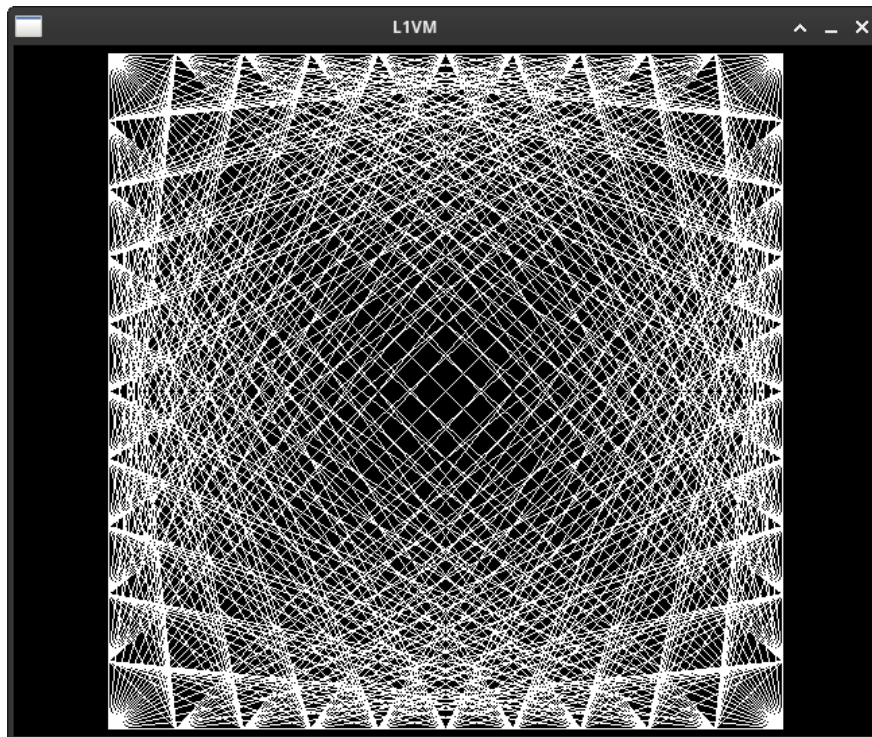
```

34 The SDL module

With the SDL module a window can be opened and graphics/GUI can be shown.

Here I want to show a simple lines demo. It draws a line pattern into the window:

34.1 Lines



Here is the program:

```

// lines.ll.com
// a draw lines program
//
#include <intr.llh>
(main func)
  (set int64 1 zero 0)
  (set int64 1 one 1)
  (set int64 1 two 2)
  (set int64 1 three 3)
  (set int64 1 width 640Q)
  (set int64 1 height 512Q)

```

```

(set int64 1 bit 32)
(set int64 1 ret 0)
(set int64 1 posstep 50)
(set int64 1 xpos 70)
(set int64 1 ypos 6)
(set int64 1 xmax 570)
(set int64 1 ymax 506)
(set int64 1 f 0)
(set int64 1 delay 1000)
(set int64 1 x 0)
(set int64 1 y 0)
(set int64 1 mouse_button_left 0)
(set int64 1 mouse_button_middle 0)
(set int64 1 mouse_button_right 0)
(set int64 1 max_delay 20)
(set int64 1 dsec 1)
(set string s sdl_errorstr "SDL: ERROR can't open screen!")
(set string s sdl_okstr "SDL: screen opened")
(set string s sdl_finishedstr "SDL: program successfully finished!")
// open screen
(zero width height bit zero :sdl_open_screen !)
(ret stpopi)
(loadreg)
(((ret zero !=) f =) f if+)
  print_s (sdl_errorstr)
  print_n
  free_mod (zero)
  exit (one)
(else)
  print_s (sdl_okstr)
  print_n
(endif)
// draw gfx
(:draw_ptop !)
// ! sdl update
(:sdl_update !)
(:wait_loop)
detime (delay)
(((dsec max_delay <) f =) f if)
  // do sdl update
  (:sdl_update !)
  (:sdl_get_mouse_state !)
  (x stpopi)
  (y stpopi)
  (mouse_button_left stpopi)
  (mouse_button_middle stpopi)
  (mouse_button_right stpopi)
  ((dsec one +) dsec =)
  (((mouse_button_left one ==) f =) f if)
    (:wait_loop jmp)
  (endif)
(endif)
(:end)
(:sdl_quit jsr)
// close sdl module
free_mod (zero)
print_s (sdl_finishedstr)
print_n
exit (zero)
(funcend)

```

```

(draw_ptop func)
  (set byte 1 col 255)
  (ypos y =)
  (((y ymax <=) f =) f for)
    (xpos x =)
    (((x xmax <=) f =) f for)
      (x ypos xmax y col col col col :sdl_line !)
      ((x posstep +) x =)
    (next)
  ((y posstep +) y =)
(next)
(ymax y =)
(((y ypos >=) f =) f for)
  (xmax x =)
  (((x xpos >=) f =) f for)
    (x ymax xpos y col col col col :sdl_line !)
    ((x posstep -) x =)
  (next)
  ((y posstep -) y =)
(next)
(ypos y =)
(((y ymax <=) f =) f for)
  (xmax x =)
  (((x xpos >=) f =) f for)
    (x ypos xpos y col col col col :sdl_line !)
    ((x posstep -) x =)
  (next)
  ((y posstep +) y =)
(next)
(ymax y =)
(((y ypos >=) f =) f for)
  (xpos x =)
  (((x xmax <=) f =) f for)
    (x ymax xmax y col col col col :sdl_line !)
    ((x posstep +) x =)
  (next)
  ((y posstep -) y =)
(next)
(funcend)
#include <sdl-lib.l1h>

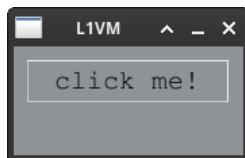
```

The lines are drawn by the “draw_ptop” function. There are four loops for the four edges.

The program will exit after the left mouse button is pressed and hold.

34.2 GUI button

Here is a second example: “prog/sdl-button.l1com”. It opens a window and shows a button. The event loop checks if the button was clicked and exits the program.



```

#include <intr-func.l1h>
(main func)
  (set const-int64 1 zero 0)
  (set const-int64 1 one 1)
  // screen width and height
  (set const-int64 1 width 170)

```

```

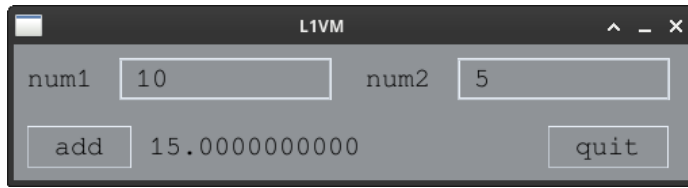
(set const-int64 1 height 80)
(set const-int64 1 width_b 169)
(set const-int64 1 height_b 79)
(set const-int64 1 bit 32)
(set const-int64 1 gadgets 1)
// font and background color
(set const-int64 1 fontsize 20)
(set string s fontname "fonts/free/FreeMono.ttf")
(set byte 1 back_r 143)
(set byte 1 back_g 147)
(set byte 1 back_b 151)
(set byte 1 alpha 255)
(set byte 1 black 0)
(set int64 1 ret 0)
(set int64 1 f 0)
// button gadget text
(set string s buttonstr "click me!")
// gadget position
(set const-int64 1 x 10)
(set const-int64 1 y 10)
// set status of button gadget: 1 = gadget selectable, 0 = gadget not active
(set int64 1 status 1)
(set string 256 retstr "")
(set int64 1 value 0)
(set int64 1 gadget_sel 0)
(set string s screenopenerrstr "ERROR: can't open screen!")
// open screen
(reset-reg)
(zero width height bit zero :sdl_open_screen !)
(ret stpopi)
// check if screen was opened
(((ret zero !=) f =) f if)
  (screenopenerrstr :print_s !)
  (:print_n !)
  (one :exit !)
(endif)
// load font
(fontnameaddr fontsize :sdl_font_ttf !)
// screen is opened, draw background color
(zero zero width_b height_b back_r back_g back_b alpha :sdl_rectangle_fill !)
// allocate space for one gadget
(gadgets :init_gui !)
// set button gadget
(zero x y buttonstr status :set_gadget_button !)
(:loop)
// wait till button gets clicked
(retstr :gadget_event !)
// get return values
(ret stpopi)
(value stpopi)
(gadget_sel stpopi)
(((gadget_sel zero ==) f =) f if)
  // user clicked button, clean up and exit
  (:sdl_free_all_gadgets !)
  (zero :exit !)
(endif)
(:loop jmp)
(funcend)
#include <sdl-lib.11h>

```

34.3 GUI Add two numbers

Here is an example which shows how to get the input of string gadgets.

The program adds two numbers and shows the result.



Here is the code with comments:

```
// sdl-math.llcom
// GUI math demo
// Add two numbers
#include <intr-func.llh>
#include <math-const.llh>
(main func)
    (set const-int64 1 zero 0)
    (set const-int64 1 one 1)
    (set const-int64 1 two 2)
    (set const-int64 1 three 3)

    // screen width and height
    (set const-int64 1 width 500)
    (set const-int64 1 height 100)
    (set const-int64 1 width_b 499)
    (set const-int64 1 height_b 99)
    (set const-int64 1 bit 32)
    (set const-int64 1 gadgets 4)

    // font and background color
    (set const-int64 1 fontsize 20)
    (set const-string s fontname "fonts/free/FreeMono.ttf")
    (set const-byte 1 back_r 143)
    (set const-byte 1 back_g 147)
    (set const-byte 1 back_b 151)
    (set const-byte 1 alpha 255)
    (set const-byte 1 black 0)
    (set int64 1 ret 0)
    (set int64 1 f 0)

    // set status of button gadget: 1 = gadget selectable, 0 = gadget not active
    (set int64 1 status 1)
    (set string 256 retstr "")
    (set int64 1 value 0)
    (set int64 1 gadget_sel 0)
    (set const-string s screenopenerrstr "ERROR: can't open screen!")

    // set first number gadget
    (set const-int64 1 x_num_one 10)
    (set const-int64 1 y_num_one 10)
    (set const-string s num_one_str "num1")
    (set const-string 11 num_one_valustr "")
    (set const-int64 1 num_one_visible 10)

    // set second number gadget
    (set const-int64 1 x_num_two 260)
```

```

(set const-int64 1 y_num_two 10)
(set const-string s num_two_str "num2")
(set const-string 11 num_two_valustr "")
(set const-int64 1 num_two_visible 10)

// set add button gadget
(set const-int64 1 x_add 10)
(set const-int64 1 y_add 60)
(set const-string s addstr "add")

// set quit button gadget
(set const-int64 1 x_quit 395)
(set const-int64 1 y_quit 60)
(set const-string s quitstr "quit")

(set double 1 num_oned 0.0)
(set double 1 num_twod 0.0)
(set double 1 resd 0.0)
(set string 256 resstr "")
(set const-int64 1 reslen 255)
(set const-int64 1 x_res 100)
(set const-int64 1 y_res 65)
(set const-int64 1 width_res 394)
(set const-int64 1 height_res 100)
(set const-string s zerostr ".0")
(set const-string s dotstr ".")

// open screen
(reset-reg)
(zero width height bit zero :sdl_open_screen !)
(ret stpopi)
// check if screen was opened
(((ret zero !=) f =) f if)
  (screenopenerrstr :print_s !)
  (:print_n !)
  (one :exit !)
(endif)

// init the modules
(one :math_init !)
(two :string_init !)

// load font
(fontnameaddr fontsize :sdl_font_ttf !)
// screen is opened, draw background color
(zero zero width_b height_b back_r back_g back_b alpha :sdl_rectangle_fill !)
// allocate space for one gadget
(gadgets :init_gui !)
// set first number gadget
(zero x_num_one y_num_one num_one_str num_one_valustr num_one_visible
status :set_gadget_string !)
// set second number gadget
(one x_num_two y_num_two num_two_str num_two_valustr num_two_visible
status :set_gadget_string !)

// set add button gadget
(two x_add y_add addstr status :set_gadget_button !)

// set quit button gadget
(three x_quit y_quit quitstr status :set_gadget_button !)

```



```

(loop)
// wait till button gets clicked
(retstr :gadget_event !)
// get return values
(ret stpopi)
(value stpopi)
(gadget_sel stpopi)
(((gadget_sel zero ==) f =) f if)
(reset-reg)
// get number one
(retstr dotstr :string_search !)
(ret stpop)
(((ret zero <) f =) f if)
// no "." decimal dot found. Add ".0"
(retstr zerostr :string_cat !)
(endif)
(retstr :string_to_double !)
(num_oned stpopd)
(endif)
(((gadget_sel one ==) f =) f if)
(reset-reg)
// get number two
(retstr dotstr :string_search !)
(ret stpop)
(((ret zero <) f =) f if)
// no "." decimal dot found. Add ".0"
(retstr zerostr :string_cat !)
(endif)
(retstr :string_to_double !)
(num_twod stpopd)
(endif)
(((gadget_sel two ==) f =) f if)
(reset-reg)
(num_oned + num_twod resd :=)

(resd :print_d !)
(:print_n !)
(:print_n !)

// convert double number to string
(resd resstr reslen :string_doubletostring !)

// clear background in the result text area
(x_res y_res width_res height_res back_r back_g back_b alpha :sdl_rectangle_fill !)

// show result text
(x_res y_res black black black resstr :sdl_text_ttf !)
(endif)
(((gadget_sel three ==) f =) f if)
// user clicked quit button, clean up and exit
(:sdl_free_all_gadgets !)
(zero :exit !)
(endif)
(loop jmp)
(funcend)
#include <sdl-lib.l1h>
#include <math-lib.l1h>
#include <string.l1h>

```

35 The FANN module

With the FANN (Fast Artificial Neural Network) module neural networks can be created, run, read and saved. (<https://leenissen.dk/>)

35.1 XOR Function

Here is the truth table for a XOR logical function:

```
0 0 : 0
0 1 : 1
1 0 : 1
1 1 : 0
```

The first two numbers are the input and the last is the output. Here “0” is false and “1” is true.

The example program uses a data file “xor.data” to set the truth table:

```
4 2 1
-1 -1
-1
-1 1
1
1 -1
1
1 1
-1
```

The first line in “xor.data” sets four data lines, two inputs and one output. Here “-1” is false and “1” is true. The second line sets the two inputs: “-1 -1” and the third line the output: “-1”.

This continues till the end.

35.2 The example

Here is the example program. It uses pointers to set the data.

```
// train-fann-file.llcom
// fann train library demo
// train XOR function and run trained neural network
#include <intr-func.llh>
(main func)
  (set const-int64 1 zero 0)
  (set const-int64 1 one 1)
  (set int64 1 i 50000)
  (set int64 1 f 0)
  (set int64 1 ret 0)
  (set const-int64 1 fann_mod 0)
  (set double 2 input1 -1.0 -1.0)
  (set double 2 input2 -1.0 1.0)
  (set double 2 input3 1.0 -1.0)
  (set double 2 input4 1.0 1.0)
  (set double 1 des_output_null -1.0)
  (set double 1 des_output_one 1.0)
  (set double 1 output 1.0)
  (set const-int64 1 Pinput1_addr 0)
  (set const-int64 1 Pinput2_addr 0)
  (set const-int64 1 Pinput3_addr 0)
  (set const-int64 1 Pinput4_addr 0)
  (set const-int64 1 Pdes_output_null_addr 0)
  (set const-int64 1 Pdes_output_one_addr 0)
  (set const-int64 1 Poutput_addr 0)
```

```

(set string 16 learning_str "learning xor...")
(set string 10 txt_1_str "0 xor 0: ")
(set string 10 txt_2_str "0 xor 1: ")
(set string 10 txt_3_str "1 xor 0: ")
(set string 10 txt_4_str "1 xor 1: ")
(set string 11 run_str "running...")
(set int64 1 handle 0)
(set int64 1 three 3)
(set int64 1 two 2)
(set int64 1 layers 3)
(set int64 1 inputs 2)
(set int64 1 outputs 1)
(set int64 1 hidden_neurons 3)
(set string s create_str "creating xor fann...")
(set string s fann_data_file_str "fann/xor.data")
(set int64 1 max_epochs 500000)
(set int64 1 epochs_between_reports 1000)
(set double 1 desired_error 0.001)

// set pointers
(input1 Pinput1_addr pointer)
(input2 Pinput2_addr pointer)
(input3 Pinput3_addr pointer)
(input4 Pinput4_addr pointer)
(des_output_null Pdes_output_null_addr pointer)
(des_output_one Pdes_output_one_addr pointer)
(output Poutput_addr pointer)

// init fann library
(reset-reg)
(fann_mod one :fann_init_lib !)
(ret stpopi)
(((ret zero !=) f =) f if)
  (one :exit !)
(endif)
(reset-reg)

// create fann
(create_str :print_s !)
(:print_n !)
(layers inputs hidden_neurons outputs :fann_create !)
(handle stpop)

// train fann from xor data file
      (handle fann_data_file_str max_epochs epochs_between_reports
desired_error :fann_train_ann_file !)

// run ann
(run_str :print_s !)
(:print_n !)
// run fann with one output variable
(handle Pinput1_addr Poutput_addr two one :fann_run !)
(txt_1_str :print_s !)
(output :print_d !)
(:print_n !)

(handle Pinput2_addr Poutput_addr two one :fann_run !)
(txt_2_str :print_s !)
(output :print_d !)
(:print_n !)

```

```

(handle Pinput3_addr Poutput_addr two one :fann_run !)
(txt_3_str :print_s !)
(output :print_d !)
(:print_n !)

(handle Pinput4_addr Poutput_addr two one :fann_run !)
(txt_4_str :print_s !)
(output :print_d !)
(:print_n !)

(handle :fann_free !)
(:free_fanns !)
(fann_mod :free_mod !)
(zero :exit !)
(funcend)
#include <fann-lib.llh>

```

Here is the output:

```

$ llvm lib/train-fann-file -q
creating xor fann...
Max epochs   500000. Desired error: 0.0010000000.
Epochs      1. Current error: 0.2862125635. Bit fail 2.
Epochs     18. Current error: 0.0006410981. Bit fail 0.
running...
0 xor 0: -0.9708444476
0 xor 1: 0.9566590190
1 xor 0: 0.9633838534
1 xor 1: -0.9647938609

```

It shows the learned outputs. Here “-0.9” is false and “0.9” is true.

You can train and run much more complex FANNs!

35.3 Cells

With Cells multiple FANNs can be connected into layers. There can be connections set between the outputs of one layer to another layer inputs. The FANNs can be run with a special command, which runs through all layers.

35.4 Here is an example

This example loads an “xor”, “or” and “and” into a Cell. The output of the “xor” and “or” are in layer “0”. The “and” in a layer above, getting the result of the others as inputs.

```

// cells-lib-2.llcom
// Cells FANN wrapper library for LLVM
//
// "xor", "or", "and" networks demo
//
#include <intr-func.llh>
(main func)
  (set const-int64 1 zero 0)
  (set const-int64 1 one 1)
  (set const-int64 1 two 2)
  (set int64 1 ret 0)

```

```

(set const-int64 1 max_cells 1)
(set int64 1 cell 0)
(set const-int64 1 node_xor 0)
(set const-int64 1 node_or 1)
(set const-int64 1 node_and 2)
(set const-int64 1 inputs 2)
(set const-int64 1 outputs 1)
(set const-int64 1 max_layers 1)
(set const-int64 1 cell_neurons 3)
(set const-int64 1 links 1)

// xor
(set double 2 node_xor_inputsf 1.0 0.0)
(set double 1 node_xor_outputsf 0.0)
(set const-int64 1 Pnode_xor_inputsfaddr 0)
(set const-int64 1 Pnode_xor_outputsfaddr 0)

// or
(set double 2 node_or_inputsf 1.0 0.0)
(set double 1 node_or_outputsf 0.0)
(set const-int64 1 Pnode_or_inputsfaddr 0)
(set const-int64 1 Pnode_or_outputsfaddr 0)

// and
(set double 2 node_and_inputsf 0.0 0.0)
(set double 1 node_and_outputsf 0.0)
(set const-int64 1 Pnode_and_inputsfaddr 0)
(set const-int64 1 Pnode_and_outputsfaddr 0)

(set double 1 outputf 0.0)
(set const-int64 1 Poutputfaddr 120)
(set const-int64 1 layer_zero 0)
(set const-int64 1 layer_one 1)
(set double 1 zerof 0.0)
(set double 1 onef 1.0)
(set int64 1 offset 8)
(set int64 1 f 0)
(set int64 1 max_layer_readout 0)
(set const-int64 1 Pmax_layer_readoutaddr 0)
// set path to "xor_float.net" file of Cells C library:
(set string s node_xor_filestr "fann/xor_float.net")
(set string s node_or_filestr "fann/or_float.net")
(set string s node_and_filestr "fann/and_float.net")

(set string s and_outputstr "AND layer output: ")
(set int64 1 ret_code 0)

// set pointers
// xor
(node_xor_inputsf Pnode_xor_inputsfaddr pointer)
(node_xor_outputsf Pnode_xor_outputsfaddr pointer)

// or
(node_or_inputsf Pnode_or_inputsfaddr pointer)
(node_or_outputsf Pnode_or_outputsfaddr pointer)

// and
(node_and_inputsf Pnode_and_inputsfaddr pointer)
(node_and_outputsf Pnode_and_outputsfaddr pointer)

```

```

(max_layer_readout Pmax_layer_readoutaddr pointer)
(outputf Poutputfaddr pointer)

(zero max_cells :cells_init_lib !)
(ret stpop)
(((ret zero !=) f =) f if)
    (one :exit !)
(endif)

// allocate memory for cell neurons
(max_cells cell_neurons :cells_alloc_neurons_equal !)
(ret stpop)
(((ret zero !=) f =) f if)
    (:cells_dealloc !)
    (loadreg)
    (one :exit !)
(endif)

// layer 0 of Cell: xor and or
// read xor FANN from file
    (cell node_xor node_xor_filestraddr inputs outputs Pnode_xor_inputsfaddr
Pnode_xor_outputsfaddr layer_zero one :cells_fann_read_ann !)
    (ret stpop)
    (((ret zero !=) f =) f if)
        // dealloc neurons
        (max_cells :cells_dealloc_neurons !)
        (ret stpop)
        (:cells_dealloc !)
        (one :exit !)
    (endif)

// read or FANN file
    (cell node_or node_or_filestraddr inputs outputs Pnode_or_inputsfaddr Pnode_or_out-
putsfaddr layer_zero one :cells_fann_read_ann !)
    (ret stpop)
    (((ret zero !=) f =) f if)
        // dealloc neurons
        (max_cells :cells_dealloc_neurons !)
        (ret stpop)
        (:cells_dealloc !)
        (one :exit !)
    (endif)

// layer 1 of Cell: and
// read and FANN file
    (cell node_and node_and_filestraddr inputs outputs Pnode_and_inputsfaddr
Pnode_and_outputsfaddr layer_one one :cells_fann_read_ann !)
    (ret stpop)
    (((ret zero !=) f =) f if)
        // dealloc neurons
        (max_cells :cells_dealloc_neurons !)
        (ret stpop)
        (:cells_dealloc !)
        (one :exit !)
    (endif)

// allocate memory for links
    (cell node_xor links :cells_alloc_node_links !)
    (ret stpop)
    (((ret zero !=) f =) f if)

```

```

        // dealloc neurons
        (max_cells :cells_dealloc_neurons !)
        (ret stpop)
        (:cells_dealloc !)
        (one :exit !)
    (endif)

    (cell node_or links :cells_alloc_node_links !)
    (ret stpop)
    (((ret zero !=) f =) f if)
        // dealloc neurons
        (max_cells :cells_dealloc_neurons !)
        (ret stpop)
        (:cells_dealloc !)
        (one :exit !)
    (endif)

    // set links
    // connect output of "xor" and "or" with the inputs of "and"
    (cell node_xor zero node_and zero zero :cells_set_node_link !)
    (ret stpop)
    (((ret zero !=) f =) f if)
        (one ret_code :=)
        (:end jmp)
    (endif)

    (cell node_or zero node_and one zero :cells_set_node_link !)
    (ret stpop)
    (((ret zero !=) f =) f if)
        (one ret_code :=)
        (:end jmp)
    (endif)

    // run the Cell FANNs
    (cell zero zero zero max_layers :cells_fann_run_ann_go_links !)
    (ret stpop)
    (((ret zero !=) f =) f if)
        (one ret_code :=)
        (:end jmp)
    (endif)

    // get output
    (cell node_and zero Pnode_and_outputsfaddr :cells_fann_get_output !)
    (ret stpop)
    (((ret zero !=) f =) f if)
        (one ret_code :=)
        (:end jmp)
    (endif)

    // print output
    (and_outputstr :print_s !)
    (node_and_outputsf :print_d !)
    (:print_n !)

    (:end)
    // cleanup
    // dealloc links
    (cell node_xor :cells_dealloc_node_links !)
    (ret stpop)
    (cell node_or :cells_dealloc_node_links !)

```

```

    (ret stpop)
    // dealloc neurons
    (max_cells :cells_dealloc_neurons !)
    (ret stpop)
    (:cells_dealloc !)
    (ret _code :exit !)
(funcend)
#include <cells-lib.llh>

```

Here is the output:

```

$ llvm cells-lib-2 -q
AND layer output: 0.9585120678

```

36 The MPFR math module

In the MPFR math module numbers with high precision can be used. The number size is only limited by the RAM in the machine. There are lots of math functions included. And numbers can be compared. They can be converted to strings to print them.

36.1 Here is an example

```

// mpfr-lib-auto.llcom
// mpfr math library
//
#include <intr-func.llh>
(main func)
  (set int64 1 zero 0)
  (set int64 1 xaddr 0)
  (set int64 1 yaddr 1)
  (set int64 1 resaddr 2)
  (set int64 1 outaddr 3)
  (set string s xstr "0.0")
  (set string s ystr "42.0")
  (set string s resstr "0.0")
  (set string s formatstr "%.20Ff")
  (set string 512 outstr "")
  (set int64 1 outstr_len 511)
  (set int64 1 one 1)
  (set int64 1 base 10)
  (set int64 1 precision 50)
  (set int64 1 f 0)
  (zero :mp_math_init call)
  // set variables
  (xstraddr base xaddr :mp_set_float !)
  // set pi
  (precision xaddr :mp_get_pi !)
  (ystraddr base yaddr :mp_set_float !)
  (resstraddr base resaddr :mp_set_float !)
  (xaddr yaddr resaddr :mp_mul_float !)
  (resaddr formatstraddr precision :mp_print_float !)
  (:print_n !)
  // save number to string
  (resaddr outstraddr outstr_len formatstr precision :mp_prints_float !)
  (outstr :print_s !)
  (:print_n !)
  // read from string and print as nuber
  (outstraddr base outaddr :mp_set_float !)
  (outaddr formatstraddr precision :mp_print_float !)

```



```

    (:print_n !)
    // check mp_greater call 42.0 > 3.14
    (yaddr xaddr :mp_greater !)
    (f stpopi)
    (f :print_i !)
    (:print_n !)
    (:mp_cleanup !)
    (zero :free_mod !)
    (zero :exit !)
(funcend)
#include <mpfr-lib-auto.llh>

```

The output:

```

$ llvm mpfr-lib-auto -q
131.946891450771317977341823279857635498046875000000000
131.946891450771317977341823279857635498046875000000000
131.946891450771317977341823279857635498046875000000000
1

```

I also wrote example code to calculate circles. It is included in the LLVM repository.

37 The vector math module

With this module math operations like: +, -, * and / can be run on arrays.

It is possible to calculate with a range of array elements. So this can be run in multiple threads on the same arrays. There are functions to do calculations with one array and a number. Which is for example added to the set array elements. And there are functions to calculate with two arrays and store the result in a third array.

37.1 Here is an example

```

// math-lib-vect.llcom
#include <intr-func.llh>
(main func)
  (set const-int64 1 zero 0)
  (set const-int64 1 max 9)
  (set int64 10 av 1 2 3 4 5 6 7 8 9 10)
  (set int64 10 bv 2 4 6 8 10 12 14 16 18 20)
  (set int64 10 cv)
  (set int64 1 Pav 0)
  (set int64 1 Pbv 0)
  (set int64 1 Pcv 0)
  (set int64 1 i 0)
  (set const-int64 1 offset 8)
  (set int64 1 real_ind 0)
  (set int64 1 f 0)
  (set int64 1 x 0)
  (set int64 1 one 1)

  (zero :math_vect_init !)

  (av Pav pointer)
  (Pav + zero Pav :=)

  (bv Pbv pointer)
  (Pbv + zero Pbv :=)

```

```

(cv Pcv pointer)
(Pcv + zero Pcv :=)

(Pav Pbv zero max Pcv :vmath_mul_int_array !)

// print cv array
(:loop)
(cv [ real_ind ] x =)
(x :print_i !)
(:print_n !)
((real_ind offset +) real_ind :=)
((i one +) i :=)
(((i max <=) f :=) f if)
(:loop jmp)
(endif)

// close module
(zero :free_mod !)
(zero :exit !)
(funcend)
#include <math-lib-vect.llh>

```

And the output:

```

$ llvm math-lib-vect -q
2
8
18
32
50
72
98
128
162
200

```

38 The net module

This is the TCP/IP network module. It can encrypt the data send with SSL/TLS sockets too.

The module has functions to create servers and clients. It is one of the more complex modules in the L1VM. To know more about it I recommend to take a look at the webserver code. It uses multithreading and memory isolation.

38.1 Server example

```
// net-lib.l1com
//
// network library for TCP/IP sockets
//
// demo waits for a telnet connection on port 3000
//
#include <intr-func.l1h>
(main func)
  (set int64 1 zero 0)
  (set int64 1 one 1)
  (set int64 1 server 0)
  (set int64 1 accept 0)
  (set int64 1 portnum 3000Q)
  (set int64 1 error 0)
  (set int64 1 server_open 0)
  (set int64 1 server_accept 0)
  (set int64 1 f 0)
  (set string 10 ip "127.0.0.1")
  (set int64 1 ip_len 10)
  (set int64 1 ret 0)
  (set string 265 bufaddr "")
  (set string 256 buf_str "")
  (set int64 1 bufsize 255)
  (set int64 1 socket_handle 0)
  (set int64 1 mod_net 0)
  (set int64 1 char 0)
  (set string 11 waiting "waiting...")
  (set string s netinitstr "error: can't init networking sockets!")
  (mod_net one :net_init !)
  (ret stpopi)
  (((ret zero !=) f =) f if)
  // ERROR can't allocate memory
  (netinitstr :print_s !)
  (:print_n !)
  (one :exit !)
(endif)
(ipaddr bufaddr :get_hostbyname !)
(ret stpopi)
// (:end jmp)
((ret zero !=) f =)
(f :try_hostbyaddr jmp)
(:open_socket jmp)
(:try_hostbyaddr)
(ipaddr bufaddr :get_hostbyaddr !)
(ret stpopi)
((ret zero !=) f =)
(f :end jmp)
(:open_socket)
(portnum :print_i !)
(:print_n !)
(bufaddr portnum :open_server_socket !)
```

```

(ret stpopi)
(socket_handle stpopi)
(bufaddr :print_s !)
(:print_n !)
(waiting :print_s !)
(:print_n !)
(socket_handle :open_accept_server !)
(ret stpopi)
(server_accept stpopi)
(portnum :print_i !)
(:print_n !)
(:wait_conn)
(server_accept buf_str bufsize :socket_read_string !)
(ret stpopi)
(((ret zero !=) f =) f if)
  (:wait_conn jmp)
(endif)
(buf_str :print_s !)
(:print_n !)
(server_accept :close_accept_server !)
(socket_handle :close_server_socket !)
(:print_n !)
(:socket_free_mem !)
(mod_net :free_mod !)
(:end)
(zero :exit !)
(funcend)
#include <net-lib.l1h>

```

This program waits on port 3000 and prints a sent string.

38.2 The webserver

Here is the webserver example. It uses the functions for mimetype getting and file sending from the net module. It is a good example about how to use multithreading and a global mutex.

```

// webserver-thread-local-2.l1com
//
// simple http webserver
// example call:
// llvm prog/webserver-thread-local -args 127.0.0.1 2000
// for localhost port 2000
//
// Use new thread for socket connection of client
//
#include <intr-func.l1h>
#include <misc-macros.l1h>
#include <math-const.l1h>
(main func)
  (set int64 1 zero 0)
  (set int64 1 one 1)
  (set int64 1 two 2)
  (set int64 1 ret 0)
  (set int64 1 maxsockets 10)
  (set int64 1 f 0)
  (set int64 1 server 0)
  (set int64 1 portnum 2000)
  (set int64 1 server_open 0)

```

```

(set int64 1 server_accept 0)
(set int64 1 ip_len 10)
(set string s filenamestr "")
(set string 256 buf "")
(set int64 1 bufsize 255)
(set int64 1 buflen 0)
(set int64 1 socket_handle 0)
(set string s netinitstr "error: can't init networking sockets!")
(set string s erroropenstr "error: can't open accept server socket!")
(set string s errorserverstr "error: can't open server socket!")
(set string 11 waiting "waiting...")
(set string s args_errstr "args: <ip> <port>")
(set int64 1 lab_run 0)
(set int64 1 cpu_number 0)
(set int64 1 fcpus 0)
(set int64 1 delay_sec 1000)
(set int64 1 delay_thr 100)
(set int64 1 thr_sync 0)
(set string 256 arg_ipstr "")
(set string 256 arg_portstr "")
(set int64 1 argnum 0)
(set int64 1 args 2)
(zero :math_init !)

shell_args (argnum)
(((argnum args !=) f =) f if)
  (args_errstr :print_s !)
  (:print_n !)
  (one :exit !)
(endif)
// get cli args
(zero arg_ipstr :get_shell_arg !)
  (one arg_portstr :get_shell_arg !)
  (arg_portstr portnum :string_to_int !)

(one maxsockets :net_init !)
(ret stpopi)
(((ret zero !=) f =) f if)
  // ERROR can't allocate memory
  (netinitstr :print_s !)
  (:print_n !)
  (one :exit !)
(endif)
(two :string_init !)
(loadreg)
(arg_ipstr bufaddr :get_hostbyname !)
(ret stpopi)
((ret zero !=) f =)
(f :try_hostbyaddr jmp)
(:open_socket jmp)
(:try_hostbyaddr)
(arg_ipstr :get_hostbyaddr !)
(ret stpopi)
((ret zero !=) f =)
(f :end jmp)
(:open_socket)
(bufaddr portnum :open_server_socket !)
(ret stpopi)
(socket_handle stpopi)
(((ret zero !=) f =) f if)

```

```

        (errorserverstr :print_s !)
        (:print_n !)
        (:wait_conn jmp)
    (endif)
    // save run function label
    (:run lab_run loadl)
    ((lab_run zero +) lab_run =)
    (:wait_conn)
    (waiting :print_s !)
    (:print_n !)
    (socket_handle :open_accept_server !)
    (ret stpopi)
    (server_accept stpopi)
    (loadreg)
    (((ret zero !=) f =) f if)
    (erroropenstr :print_s !)
    (:print_n !)
    (:wait_conn jmp)
    (endif)
    // (socket_handle server_accept :run !)
    (cpu_number :get_cpu !)
    pull_int64_var (cpu_number)
    (((cpu_number zero ==) f =) f if)
    (:wait_run)
    (fcpus :free_cpus !)
    (fcpus :pull_int64_var !)
    (((fcpus one >) f =) f if+)
    (socket_handle stpushi)
    (server_accept stpushi)
    // set sync flag to zero
    (zero :globallock !)
    (zero thr_sync =)
    (zero :globalunlock !)
    (lab_run :thread !)
    // wait till thread got the stack values and set sync flag to one!
    (:wait_thr)
    (delay_thr :detime !)
    (zero :globallock !)
    (((thr_sync one !=) f =) f if)
    (zero :globalunlock !)
    (:wait_thr jmp)
    (endif)
    (zero :globalunlock !)
    // get values of current main thread stack
    (ret stpopi)
    (ret stpopi)
    (else)
    (delay_sec :detime !)
    (:wait_run jmp)
    (endif)
    (endif)
    // (((cpu_number zero !=) f =) f if)
    // join
    // (endif)
    // do loop
    (:wait_conn jmp)
    (:end)
    (:socket_free_mem !)
    (zero :exit !)
(funcend)

```

```

(run func)
  #var ~ run
  (set int64 1 server~ 0)
  (set int64 1 accept~ 0)
  (set int64 1 zero~ 0)
  (set int64 1 two~ 2)
  (set int64 1 three~ 3)
  (set int64 1 four~ 4)
  (set int64 1 five~ 5)
  (set int64 1 server_act_open~ 1)
  (set int64 1 shutdown~ 0)
  (set int64 1 err~ 0)
  (set int64 1 err_ok~ 0)
  (set int64 1 len~ 0)
  (set int64 1 i~ 0)
  (set int64 1 filehandle~ 0)
  (set int64 1 filesize~ 255)
  (set int64 1 n~ 0)
  (set int64 1 ret~ 0)
  (set int64 1 f~ 0)
  (set int64 1 stringlen~ 255)
  (set int64 1 slen~ 0)
  (set string 256 comm~)
  (set string s comm_get~ "GET")
  (set string 2 ch~)
  (set string 256 filename~ "")
  (set string 256 filesend~)
  (set string 256 header~)
  (set string s space~ " ")
  (set string 256 read~ "r")
  (set string 256 mimetype~)
  (set string s emptystr~ "")
  (set int64 1 mimetype_len~ 255)
  (set string 256 buf~ "")
  (set string s error_clientaddr~ "webserver run ERROR: can't get client address!")
  (set string s error_sread~ "webserver run ERROR: can't read request!")
  (set string s file_req~ "Got file request: ")
  (set string s error_mimetype~ "webserver: ERROR can't get file mimetype!")
  (set string s error_send_file~ "webserver: ERROR can't send file!")
  (set string s error_get_filename~ "webserver: ERROR can't get filename!")
  (set string s file_send~ "try to send file: ")
  (set string s error_close_accept~ "ERROR: close accept socket !!")
  (set string s webserver_rootstr~ "web")
  (set string 256 filename_realstr~ "")
  // get values from stack
  (:data_func_local_alloc!)
  (:data_func_local!)
  (accept~ stpopi)
  (server~ stpopi)
  (:data_func_global!)
  // set sync flag to one
  (zero :globallock!)
  (one thr_sync=)
  (zero :globalunlock!)
  (:data_func_local!)
  //
  //
  // parse loop =====
  //
  //

```

```

(:run_parse)
(accept~ buf~ stringlen~ :socket_read_string !)
(ret~ stpopi)
(((ret~ zero~ !=) f~ =) f~ if)
  (error_clientaddr~ :print_s !)
  (:print_n !)
  (:parse_end jmp)
(endif)
// print request
(buf~ :print_s !)
(:print_n !)
// get filename
(buf~ filename~ filesize~ :socket_handle_get !)
(ret~ stpopi)
(((ret~ zero~ !=) f~ =) f~ if)
  (error_get_filename~ :print_s !)
  (:print_n !)
  (:close_accept jmp)
(endif)
(filename_realstr~ webserver_rootstr~ :string_copy !)
(filename_realstr~ filename~ :string_cat !)
(filename~ filename_realstr~ :string_copy !)
(:parse_end)
//
//
// get mimetype and send file =====
//
//
(filename~ :print_s !)
(:print_n !)
(filename~ mimetype~ mimetype_len~ :get_mimetype_from_filename !)
(ret~ stpopi)
(((ret~ zero~ !=) f~ =) f~ if)
  (error_mimetype~ :print_s !)
  (:print_n !)
(endif)
(accept~ filename~ mimetype~ :socket_send_file !)
(ret~ stpopi)
(((ret~ zero~ !=) f~ =) f~ if)
  (error_send_file~ :print_s !)
  (:print_n !)
(endif)
(:close_accept)
// close accept server
(accept~ :close_accept_server !)
(ret~ stpopi)
(((ret~ zero~ !=) f~ =) f~ if)
  (error_close_accept~ :print_s !)
  (:print_n !)
(endif)
(:data_func_global !)
(:data_func_free !)
(zero~ :threadexit !)
(funcend)
#include <math-lib.llh>
#include <net-lib.llh>
#include <string.llh>

```


39 The memory module

A standard variable allocation is done by “set”. But the variable size must be set statically as a constant. Often the needed size is known at runtime only. For this there is the memory module. The array variable size can be set at the time of allocation. The variable must be deallocated before the program exits. There are functions to allocate all variable types.

39.1 The example

```
// mem-lib.llcom
#include <intr-func.llh>
(main func)
  (set int64 1 zero 0)
  (set int64 1 one 1)
  (set int64 1 memaddr 0)
  (set int64 1 memsize 10000)
  (set int64 1 i 0)
  (set int64 1 j 0)
  (set int64 1 f 0)
  (set int64 1 read 0)
  (set int64 1 write 0)
  (set int64 1 ret 0)
  (set string s meminitstr "ERROR can't init memory")
  (set string s memerrstr "ERROR can't allocate memory!")
  (set string s writing "writing: ")
  (set string s reading "reading: ")
  (zero one :mem_init call)
  (ret stpopi)
  (loadreg)
  (((ret zero !=) f =) f if)
    // ERROR can't allocate memory
    (meminitstr :print_s !)
    (:print_n !)
  (one :exit !)
  (endif)
  // allocate int64 array
  (memsize :alloc_int64 call)
  // get memory handle memaddr:
  (memaddr stpopi)
  (((memaddr zero <) f =) f if)
    // ERROR can't allocate memory
    (memerrstr :print_s !)
  (:print_n !)
  (one :exit !)
  (endif)

  // write to array: 0 to memsize - 1
  (:write_loop)
  (writing :print_s !)
  (i :print_i !)
  (:print_n !)
  (memaddr i i :int_to_array !)
  ((i one +) i =)
  (((i memsize <) f =) f if)
    (:write_loop jmp)
  (endif)
  (:print_n !)
  (:print_n !)

  // read from array: 0 to memsize - 1
```

```

(zero j =)
(:read_loop)
(memaddr j :array_to_int call)
(read stpopi)
(reading :print_s !)
(read :print_i !)
(:print_n !)
((j one +) j =)
(((j memsize <) f =) f if)
(:read_loop jmp)
(endif)

// dealloc memory
(:free_mem !)
// close mem module
(zero :free_mod !)
// exit OK
(zero :exit !)
(funcend)
#include <mem-lib.llh>

```

40 The vector memory module

In this module vector arrays like in C can be allocated, written and read. A vector array can increase its size or reduce it dynamically at runtime. So this can be used if an array size gets modified in a program.

40.1 An example

```

// mem-lib-vect.ll com
#include <intr-func.llh>
(main func)
  (set const-int64 1 zero 0)
  (set const-int64 1 one 1)
  (set int64 1 memaddr 0)
  (set const-int64 1 memsize 5)
  (set int64 1 i 0)
  (set int64 1 j 0)
  (set int64 1 f 0)
  (set int64 1 read 0)
  (set int64 1 write 0)
  (set int64 1 ret 0)
  (set const-string s meminitstr "ERROR can't init memory")
  (set const-string s memerrstr "ERROR can't allocate memory!")
  (set const-string s writing "writing: ")
  (set const-string s reading "reading: ")
  (zero one :mem_init !)
  (ret stpop)
  (((ret zero !=) f =) f if)
    // ERROR can't allocate memory
    (meminitstr :print_s !)
    (:print_n !)
    (one :exit !)
  (endif)
  // allocate
  (:alloc_int64_vect call)
  (memaddr stpopi)
  (((memaddr zero <) f =) f if)

```

```

        // ERROR can't allocate memory
        (meminitstr :print_s !)
(:print_n !)
(one :exit !)
(endif)
// write to array
(:write_loop)

(writing :print_s !)
(:print_n !)
(i :print_i !)
(:print_n !)

(memaddr i :int_to_vect !)
((i one +) i =)
(((i memsize <) f =) f if)
        (:write_loop jmp)
(endif)

(:print_n !)
(:print_n !)

// read from array
(zero j =)
(j :print_i !)
(:print_n !)
(:read_loop)
(memaddr j :vect_to_int !)
(read stpopi)

(reading :print_s !)
(:print_n !)
(j :print_i !)
(:print_n !)

((j one +) j =)
(((j memsize <) f =) f if)
        (:read_loop jmp)
(endif)
// dealloc mem
(memaddr :dealloc_vect !)
(:free_mem !)
// close mem module
(zero :free_mod !)
// exit ok
(zero :exit !)
(funcend)
#include <mem-lib.llh>

```

41 The memory object module

In Brackets there are no structs like in C or other languages. So you have to build something like this. But there is something new I call “memory objects”. In a memory object array there can all data types be stored in the same array: “int64”, “double”, and “string”. The variable type is set at the memory object save function. The array index for the start of the save is set there too.

The variables can be get by the load function. This makes it possible to set a variable type dynamically at runtime.

41.1 Here is the example:

```
// mem-obj-lib.llcom
// memory object demo
#include <intr-func.llh>
(main func)
    (set const-int64 1 zero 0)
    (set const-int64 1 one 1)
    (set const-int64 1 two 2)
    (set const-int64 1 three 3)
    (set int64 1 memaddr 0)
    (set int64 1 memind 0)
    (set int64 1 memsize 1)
    (set int64 1 variables 4)
    (set int64 1 a 23)
    (set int64 1 b 42)
    (set double 1 c 3.1415)
    (set int64 1 aread)
    (set int64 1 bread)
    (set double 1 cread)
    (set int64 1 read 0)
    (set int64 1 write 0)
    (set int64 1 ret 0)
    (set int64 1 f 0)
    (set const-string s meminitstr "ERROR can't init memory")
    (set const-string s memerrstr "ERROR can't allocate memory!")
    (set const-string s saveerrstr "ERROR can't save to memory object!")
    (set const-string s writing "writing: ")
    (set const-string s reading "reading: ")
    (set const-string s hellostr "Hello world!")
    (set string 256 getstr "")

#include <mem-obj-lib-types.llh>

(zero one :mem_obj_init !)
(ret stpopi)
(loadreg)
(((ret zero !=) f =) f if)
    // ERROR can't allocate memory
    (meminitstr :print_s !)
(:print_n !)
(one :exit !)
(endif)
// allocate memory for the 4 variables
(variables memsize :alloc_obj_memobj !)
(memaddr stpopi)
(loadreg)
(((memaddr zero <) f =) f if)
    // ERROR can't allocate memory
    (memerrstr :print_s !)
(:print_n !)
```

```

        (one :exit !)
    (endif)
    // write to memory obj array
    // store variables a, b, c and hellostr in memory object
    // note the variable types are set by "i_type", "d_type" and "s_type" !!!
    (c d_type b i_type a i_type hellostr s_type variables zero memaddr :save_obj_memobj !)
    (ret stpopi)
    (((ret zero !=) f =) f if)
        // ERROR can't allocate memory
        (saveerrstr :print_s !)
        (:print_n !)
        (one :exit !)
    (endif)
    // get string from mem obj
    (zero memaddr getstr :load_obj_string_memobj !)
    (ret stpopi)
    (getstr :print_s !)
    (:print_n !)
    (three one memaddr :load_obj_memobj !)
    // pop variables from stack
    (ret stpopi)
    (cread stpopd)
    (bread stpopi)
    (aread stpopi)
    // print variables
    (aread :print_i !)
    (:print_n !)

    (bread :print_i !)
    (:print_n !)

    (cread :print_d !)
    (:print_n !)

    // free mem object
    (memaddr :free_obj_memobj !)
    (:free_memobj !)
    // close mem module
    (zero :free_mod !)
    // exit OK
    (zero :exit !)
(funcend)
#include <mem-obj-lib.l1h>

```

42 The message module

L1VM has a message module to send messages between program parts. This can be useful to send data to a thread for example. To synchronize things.

42.1 An example

```
// messages-mod.ll.com
// message send/receive module
#include <intr-func.llh>
#include <messages-mod-func.llh>

(main func)
  (set const-int64 1 zero 0)
  (set const-int64 1 one 1)
  (set int64 1 ret 0)
  (set int64 1 memaddr 0)
  (set int64 1 message 2021)
  (set int64 1 id 10)
  (set int64 1 message_read 0)
  (set int64 1 handle 0)
  (set int64 1 message_type 0)
  (set const-int64 1 messages_max 1)
  (set const-string s messagestr "got message: ")
  (set const-string s errsendstr "error sending message!")
  (set const-string s errstr "error searching message!")
  (set const-string s msg_typestr "message type: ")
  (set int64 1 f 0)
  (zero messages_max :msg_init !)
  (id message :i-> !)
  (ret stpop)
  (((ret zero !=) f :=) f if)
  (errsendstr :print_s !)
  (:print_n !)
  (one :exit !)
  (endif)
  (id :msg? !)
  (handle stpop)
  (((handle zero <) f :=) f if)
  (errstr :print_s !)
  (:print_n !)
  (one :exit !)
  (endif)
  (handle :->i !)
  (ret stpop)
  (message_read stpop)
  (messagestr :print_s !)
  (message_read :print_i !)
  (:print_n !)
  (handle :msgtype? !)
  (message_type stpop)
  (msg_typestr :print_s !)
  (message_type :print_i !)
  (:print_n !)

  (:msg_free !)

  (zero :exit !)
(funcend)
#include <messages-mod.llh>
```

Here is the output:

```
$ llvm messages-mod -q
got message: 2021
message type: 2
```

43 The Nano ID module

With the nano id module unique IDs can be created. There can be a custom char string set, with the wanted chars in the ID. And also the length of the ID can be set. You can use this everywhere an unique ID is needed.

43.1 Here is an example

```
// nanoid.llcom
//
// nanoid create unique ids demo
//
#include <intr-func.llh>

(main func)
  (set int64 1 zero 0)
  (set int64 1 mod_nanoid 0)
  (set string 65 idstr)
    (set string s custom_alphabetstr "abcdefghijklmnopqrstuvwxyzABCDE-
FGHIJKLMNOPQRSTUVWXYZ0123456789+-*#.:@")
  (set int64 1 custom_len 64)

  // init nanoid module
  (mod_nanoid :nanoid_init !)

  // create 21 char ID
  (idstr :nanoid_create !)
  (idstr :print_s !)
  (:print_n !)

  // set custom ID
  (idstr custom_alphabetstr custom_len :nanoid_create_custom !)
  (idstr :print_s !)
  (:print_n !)

  (zero :exit !)
(funcend)

#include <nanoid.llh>
```

And here is the output:

```
$ llvm nanoid -q
Rx4-PxAtY_Vyocm2GBkba
kQWm.GPtxzvgG@4i#k2sqtZulO:*t#bDMuPh02.Vk61n6lwzvhrj.zDi#BS4xTqH
```

44 The Process module

With the process module a new shell command can be started. You can use this to start programs or shell scripts. The return exit code can be checked. It is returned by the process call function.

NOTE: it can be dangerous to run programs which use the process call function. As the file access can not be set to the llvm root directory only! And it will be really dangerous if a shell runs with root rights! If you remove the process library from the “llvm/bin” directory then no program can use it!

44.1 Here is an example

```
// process-lib.llcom
// See process module!
#include <intr.llh>
(main func)
    (set int64 1 zero 0)
    (set int64 1 ret 0)
    (set string s procstr "ls")
    // init library
    (zero :process_init !)
    // call system process
    (procstr :run_shell !)
    (ret stpopi)
    // print return value of shell call
    print_i (ret)
    print_n
    exit (zero)
(funcend)
#include <process-lib.llh>
```

And here is the output:

```
$ llvm process-lib -q
array.llcom          mpfr-lib-auto-orig.llcom
array.lldbg          mpfr-lib-auto-orig.lldbg
array.llobj          mpfr-lib-auto-orig.llobj
<SNIP>
0
```

The exit code is “0” so there was no error.

45 The Lisp like array functions

I wrote array functions to combine two arrays into a new array.

There is also a function to return the first element of an array. Or to return the elements from “1” to the array end, this is the “rest” function.

The “cons” function inserts a variable before an array and returns a new array.

The “member” function searches a variable in an array. If it is found the array is returned. It sets then a return value of “0”. A return value of “1” means the variable is not in the array.

45.1 Here is an example

```
// array-copy.llcom

#include <intr-func.llh>
#include <misc-macros.llh>
#include <zero.llh>

(main func)
  (set int64 1 zero 0)
  (set int64 1 one 1)
  (set int64 1 four 4)
  (set int64 5 a 23 4 10 50 100)
  (set int64 5 b 0 10 11 23 55)
  (set int64 10 c)
  (set int64 4 d)
  (set int64 1 Pa 0)
  (set int64 1 Pb 0)
  (set int64 1 Pc 0)
  (set int64 1 Pd 0)
  (set int64 1 i 0)
  (set int64 1 offset 8)
  (set int64 1 f 0)
  (set int64 1 maxarray 10)
  (set int64 1 maxdarray 4)
  (set int64 1 realindex 0)
  (set int64 1 var 0)
  (set int64 1 cons_var 77)
  (set int64 1 Pcons_var 0)

  // set pointers
  (a Pa pointer)
  (Pa :pull_int64_var !)

  (b Pb pointer)
  (Pb :pull_int64_var !)

  (c Pc pointer)
  (Pc :pull_int64_var !)

  (d Pd pointer)
  (Pd :pull_int64_var !)

  (cons_var Pcons_var pointer)
  (Pcons_var :pull_int64_var !)

  (Pa Pb Pc zero four zero four :array_int64_cat_range !)

  (((i maxarray <) f :=) f for)
  (c [ realindex ] var :=)
```

```

    (var :print_i !)
    (:print_n !)

    (realindex + offset realindex :=)
    (i + one i :=)
  (next)

  (Pc :array_int64_first !)
  (var stpop)
  (:print_n !)
  (var :print_i !)
  (:print_n !)
  (:print_n !)

  (Pa Pd :array_int64_rest !)

  (zero i :=)
  (zero realindex :=)

  (((i maxdarray <) f :=) f for)
    (d [ realindex ] var :=)
    (var :print_i !)
    (:print_n !)

    (realindex + offset realindex :=)
    (i + one i :=)
  (next)

  (:print_n !)
  (:print_n !)

  (Pcons_var Pa Pc :array_int64_cons !)

  (zero i :=)
  (zero realindex :=)

  (((i maxarray <) f :=) f for)
    (c [ realindex ] var :=)
    (var :print_i !)
    (:print_n !)

    (realindex + offset realindex :=)
    (i + one i :=)
  (next)

  (zero :exit !)
(funcend)

#include <array-cat.l1h>

```

This is the output:

```
$ llvm array-copy -q
23
4
10
50
100
0
10
11
23
55

23

4
10
50
100

77
23
4
10
50
100
10
11
23
55
```

III - Assembly

46 Assembly - Simple programs

In Brackets inline assembly can be written. The assembly code is left unchanged and is put into the assembly output. Here I will show how some programs compile to assembly code.

46.1 Hello world

Let's start with something simple:

```
// hello.llcom
// Brackets - Hello world!
//
#include <intr-func.llh>
(main func)
    (set int64 1 zero 0)
    (set string 13 hello "Hello world!")
    // print string
    (hello :print_s !)
    (:print_n !)
    (zero :exit !)
(funcend)
```

The program output:

```
llvm hello -q
Hello world!
```

And here is the assembly code:

```
.data
Q, 1, zero
@, 0Q, 0
B, 13, hello
@, 8Q, "Hello world!"
Q, 1, helloaddr
@, 21Q, 8Q
.dend
.code
:main
loada zero, 0, 0
loada helloaddr, 0, 1
intr0 6, 1, 0, 0
intr0 7, 0, 0, 0
intr0 255, 0, 0, 0
rts
.cend
```

Here you can see the two sections: the data section inside “.data” and “.dend”. And the code section between “.code” and “.cend”.

Here is the definition of the int64 variable “zero”:

```
Q, 1, zero      // define zero as a “int64” (Q) variable with the size of 1 element
@, 0Q, 0        // store the variable at address “0” with the value of “0”
```

The string definition is more complex:

```
B, 13, hello      // define the string “hello” with the length of 13 chars
@, 8Q, "Hello world!" // store the string “Hello world!” at the address “8”
```

```

Q, 1, helloaddr      // define the pointer to the string variable as "int64" (Q)
@, 21Q, 8Q           // store the pointer in address "21" as "int64"

```

Here is the main function:

```

:main                // the beginning of the main function
loada zero, 0, 0      // load "zero" into int64 register "0"
loada helloaddr, 0, 1 // load the string pointer "helloaddr" into int64 register "1"
intr0 6, 1, 0, 0      // call interrupt intr0: 6 (print_s) to print string (reg 1)
intr0 7, 0, 0, 0      // call interrupt intr0: 7 (print_n) to print newline (reg 0)
intr0 255, 0, 0, 0    // call interrupt intr0: 255 (exit) to exit program with code "0"
rts

```

46.2 Fizz-Buzz

Fizz Buzz is a popular children's learning game in which players usually sit in a circle and take turns counting aloud from one upwards. All numbers divisible by three must be replaced with the word "Fizz," and all numbers divisible by five must be replaced with the word "Buzz."

If a number is divisible by three and five, then "Fizz Buzz" is called out.

This problem is often asked in job interviews. Here is the code:

```

// fizz-buzz.llcom
#include <intr-func.llh>
(main func)
    (set const-int64 1 zero 0)
    (set const-int64 1 one 1)
    (set const-int64 1 fizz 3)
    (set const-int64 1 buzz 5)
    (set const-string s fizzstr "fizz")
    (set const-string s buzzstr "buzz")
    (set const-int64 1 max 100)
    (set int64 1 i 1)
    (set int64 1 f 0)
    (set int64 1 modfizz 0)
    (set int64 1 modbuzz 0)

    (((i max <=) f :=) f for)
    (i % fizz modfizz :=)
    (i % buzz modbuzz :=)
    ((modfizz == zero) && (modbuzz == zero) f :=)
    (f if+)
    (fizzstr :print_s !)
    (buzzstr :print_s !)
    (:print_n !)
    (:skip jmp)
    (else)
    (((modfizz zero ==) f =) f if)
    (fizzstr :print_s !)
    (:print_n !)
    (:skip jmp)
    (endif)
    (((modbuzz zero ==) f =) f if)
    (buzzstr :print_s !)
    (:print_n !)
    (:skip jmp)
    (endif)
    (i :print_i !)
    (:print_n !)

```

```

        (:skip)
        (i + one i :=)
    (endif)
(next)
    (zero :exit !)
(funcend)

```

Here is the assembly output:

```

.data                                // the data definitions just like in the Hello world program
Q, 1, zero
@, 0Q, 0
Q, 1, one
@, 8Q, 1
Q, 1, fizz
@, 16Q, 3
Q, 1, buzz
@, 24Q, 5
B, 5, fizzstr
@, 32Q, "fizz"
Q, 1, fizzstraddr
@, 37Q, 32Q
B, 5, buzzstr
@, 45Q, "buzz"
Q, 1, buzzstraddr
@, 50Q, 45Q
Q, 1, max
@, 58Q, 100
Q, 1, i
@, 66Q, 1
Q, 1, f
@, 74Q, 0
Q, 1, modfizz
@, 82Q, 0
Q, 1, modbuzz
@, 90Q, 0
.dend
.code
:main                                // the main function
loada zero, 0, 0                     // load into the registers
loada fizzstraddr, 0, 1
intr0 34, 1, 0, 0
intr0 34, 1, 0, 0
loada buzzstraddr, 0, 2
intr0 34, 2, 0, 0                     // set string "buzzstraddr" as immutable
intr0 34, 2, 0, 0
:for_0                                // for loop start
loada i, 0, 1                         // load into the registers
loada max, 0, 2
lseqi 1, 2, 3                          // if
load f, 0, 4
pullqw 3, 4, 0
loada f, 0, 1
jmp 1, :for2_0
jmp :forend_0
:for2_0
loada i, 0, 2
loada fizz, 0, 3
modi 2, 3, 4                          // modulo "3"
load modfizz, 0, 5

```

```

pullqw 4, 5, 0
loada buzz, 0, 4
modi 2, 4, 5                                // modulo "5"
load modbuzz, 0, 6
pullqw 5, 6, 0
loada modfizz, 0, 5
loada zero, 0, 6
eqi 5, 6, 7
loada modbuzz, 0, 8
eqi 8, 6, 9
andi 7, 9, 10                               // and "&&"
load f, 0, 11
pullqw 10, 11, 0
loada f, 0, 1
jmp i 1, :if_0                               // jump to if = true code
jmp :else_0                                  // jump to else code
:if_0
loada fizzstraddr, 0, 2                     // load into registers
intr0 6, 2, 0, 0                           // print "fizz"
loada buzzstraddr, 0, 3
intr0 6, 3, 0, 0                           // print "buzz"
intr0 7, 0, 0, 0
jmp :skip                                   // jump to ":skip"
jmp :endif_0                               // jump to ":endif_0"
:else_0                                     // else
loada modfizz, 0, 1
loada zero, 0, 2
eqi 1, 2, 3
load f, 0, 4
pullqw 3, 4, 0                             // pull register "3" into variable "f"
loada f, 0, 1
jmp i 1, :if_1
jmp :endif_1
:if_1
loada fizzstraddr, 0, 2
intr0 6, 2, 0, 0
intr0 7, 0, 0, 0
jmp :skip
:endif_1
loada modbuzz, 0, 1
loada zero, 0, 2
eqi 1, 2, 3
load f, 0, 4
pullqw 3, 4, 0
loada f, 0, 1
jmp i 1, :if_2
jmp :endif_2
:if_2
loada buzzstraddr, 0, 2
intr0 6, 2, 0, 0
intr0 7, 0, 0, 0
jmp :skip
:endif_2
loada i, 0, 1
intr0 4, 1, 0, 0
intr0 7, 0, 0, 0
:skip
loada i, 0, 1
loada one, 0, 2
addi 1, 2, 3

```

```

load i, 0, 4
pullqw 3, 4, 0
:endif _0
jmp :for _0
:forend _0
loada zero, 0, 1
intr0 255, 1, 0, 0
.cend

```

Here is the programs output up to 20:

```

$ llvm fizz-buzz -q
1
2
fizz
4
buzz
fizz
7
8
fizz
buzz
11
fizz
13
14
fizzbuzz
16
17
fizz
19
buzz

```

46.3 Hello world - inline assembly

Here I show how inline assembly can be inserted into Brackets code. We call a function which calculates the square of a number.

Here is the code:

```

// hello-asm.llcom
// Brackets - Hello world!
#include <intr-func.llh>
(main func)
    (set int64 1 zero 0)
    (set int64 1 num 5)
    (set int64 1 num_square 0)
    (set string 13 hello "Hello world!")
    // print string
    (hello :print_s !)
    (:print_n !)

    // calc square
    (num :square !)
    (num_square stpop)
    (num_square :print_i !)
    (:print_n !)

    (zero :exit !)
(funcend)

```



```

(square func)
  (ASM)
    stpopi 1
    muli 1, 1, 2
    stpushi 2
  (ASM_END)
(funcend)

```

The assembly code is between “(ASM)” and “(ASM_END)”.

```

    stpopi 1          // pop stack int64 into register “1”
    muli 1, 1, 2      // multiply register “1” with “1” and store the result in register “2”
    stpushi 2         // push the register as int64 on the stack

```

Here is the output of the compiler:

```

.data
Q, 1, zero
@, 0Q, 0
Q, 1, num
@, 8Q, 5
Q, 1, num_square
@, 16Q, 0
B, 13, hello
@, 24Q, "Hello world!"
Q, 1, helloaddr
@, 37Q, 24Q
.dend
.code
:main
loada zero, 0, 0
loada helloaddr, 0, 1
intr0 6, 1, 0, 0
intr0 7, 0, 0, 0
stpushi 0          // save register “0” (for context switch)
stpushi 1          // save register “1”
loada num, 0, 2
stpushi 2          // push register “2” (num) on the stack
jsr :square        // call function “square”
stpopi 3           // pop stack into register “3”
load num_square, 0, 4 // load address of “num_square” into register “4”
pullqw 3, 4, 0     // pull register “3” into the address stored in register “4”
stpopi 1           // restore register “1”
stpopi 0           // restore register “0”
loada num_square, 0, 1 // load “num_square” into register “1”
intr0 4, 1, 0, 0    // interrupt 0: 4: “print_i”
intr0 7, 0, 0, 0    // interrupt 0: 7: “print_n”
loada zero, 0, 2
intr0 255, 2, 0, 0  // exit program
rts
:square            // function “square” as explained above
loada zero, 0, 0
    stpopi 1
    muli 1, 1, 2
    stpushi 2
rts
.cend

```

The assembly opcodes and their arguments are defined in “include/opcodes.h”:

```

struct opcode opcode[] =
{
    { "pushb", 3, { I_REG, I_REG, I_REG, EMPTY } },
    { "pushw", 3, { I_REG, I_REG, I_REG, EMPTY } },
    { "pushdw", 3, { I_REG, I_REG, I_REG, EMPTY } },
    { "pushqw", 3, { I_REG, I_REG, I_REG, EMPTY } },
    { "pushd", 3, { I_REG, I_REG, D_REG, EMPTY } },

    { "pullb", 3, { I_REG, I_REG, I_REG, EMPTY } },
    { "pullw", 3, { I_REG, I_REG, I_REG, EMPTY } },
    { "pulldw", 3, { I_REG, I_REG, I_REG, EMPTY } },
    { "pullqw", 3, { I_REG, I_REG, I_REG, EMPTY } },
    { "pulld", 3, { D_REG, I_REG, I_REG, EMPTY } },    // 9

    { "addi", 3, { I_REG, I_REG, I_REG, EMPTY } },
    { "subi", 3, { I_REG, I_REG, I_REG, EMPTY } },
    { "muli", 3, { I_REG, I_REG, I_REG, EMPTY } },
    { "divi", 3, { I_REG, I_REG, I_REG, EMPTY } },

    { "addd", 3, { D_REG, D_REG, D_REG, EMPTY } }, // 14
    { "subd", 3, { D_REG, D_REG, D_REG, EMPTY } },
    { "muld", 3, { D_REG, D_REG, D_REG, EMPTY } },
    { "divd", 3, { D_REG, D_REG, D_REG, EMPTY } },

    { "smuli", 3, { I_REG, I_REG, I_REG, EMPTY } }, // 18
    { "sdivi", 3, { I_REG, I_REG, I_REG, EMPTY } },

    { "andi", 3, { I_REG, I_REG, I_REG, EMPTY } },
    { "ori", 3, { I_REG, I_REG, I_REG, EMPTY } },
    { "bandi", 3, { I_REG, I_REG, I_REG, EMPTY } },
    { "bori", 3, { I_REG, I_REG, I_REG, EMPTY } },
    { "bxori", 3, { I_REG, I_REG, I_REG, EMPTY } },
    { "modi", 3, { I_REG, I_REG, I_REG, EMPTY } }, // 25

    { "eqi", 3, { I_REG, I_REG, I_REG, EMPTY } },
    { "neqi", 3, { I_REG, I_REG, I_REG, EMPTY } },
    { "gri", 3, { I_REG, I_REG, I_REG, EMPTY } },
    { "lsi", 3, { I_REG, I_REG, I_REG, EMPTY } },
    { "greqi", 3, { I_REG, I_REG, I_REG, EMPTY } },
    { "lseqi", 3, { I_REG, I_REG, I_REG, EMPTY } }, // 31

    { "eqd", 3, { D_REG, D_REG, I_REG, EMPTY } },
    { "neqd", 3, { D_REG, D_REG, I_REG, EMPTY } },
    { "grd", 3, { D_REG, D_REG, I_REG, EMPTY } },
    { "lsd", 3, { D_REG, D_REG, I_REG, EMPTY } },
    { "greqd", 3, { D_REG, D_REG, I_REG, EMPTY } },
    { "lseqd", 3, { D_REG, D_REG, I_REG, EMPTY } },

    { "jmp", 1, { LABEL, EMPTY, EMPTY, EMPTY } }, // 38
    { "jmpb", 2, { I_REG, LABEL, EMPTY, EMPTY } },

    { "stpushb", 1, { I_REG, EMPTY, EMPTY, EMPTY } },
    { "stpopb", 1, { I_REG, EMPTY, EMPTY, EMPTY } },

    { "stpushi", 1, { I_REG, EMPTY, EMPTY, EMPTY } },
    { "stpopi", 1, { I_REG, EMPTY, EMPTY, EMPTY } },

    { "stpushd", 1, { D_REG, EMPTY, EMPTY, EMPTY } },
    { "stpopd", 1, { D_REG, EMPTY, EMPTY, EMPTY } },

```

```

{ "loada", 3, { DATA, DATA_OFFS, I_REG, EMPTY }, },
{ "loadd", 3, { DATA, DATA_OFFS, D_REG, EMPTY }, },
{ "intr0", 4, { ALL, ALL, ALL, ALL }, },
{ "intr1", 4, { ALL, ALL, ALL, ALL }, },

{ "inclsijmpi", 3, { I_REG, I_REG, LABEL, EMPTY }, },
{ "decgrijmpi", 3, { I_REG, I_REG, LABEL, EMPTY }, },

{ "movi", 2, { I_REG, I_REG, EMPTY, EMPTY }, },
{ "movd", 2, { D_REG, D_REG, EMPTY, EMPTY }, },
{ "loadl", 2, { LABEL, I_REG, EMPTY, EMPTY }, },
{ "jmpa", 1, { I_REG, EMPTY, EMPTY, EMPTY }, },
{ "jsr", 1, { LABEL, EMPTY, EMPTY, EMPTY }, },
{ "jsra", 1, { I_REG, EMPTY, EMPTY, EMPTY }, },
{ "rts", 0, { EMPTY, EMPTY, EMPTY, EMPTY }, },

{ "load", 3, { DATA, DATA_OFFS, I_REG, EMPTY }, },

{ "noti", 2, { I_REG, I_REG, EMPTY, EMPTY }, }
};

```

The math opcodes have “3” arguments. The target is the third register. You can also compile programs and take a look at the generated assembly code.

If you are using inline assembly then you can use the registers directly. Without the need of using variables. There are 256 registers for int64 and for double floating point numbers.

Note: make sure not to use the int64 register “0” in your assembly code. It has to be set to zero always!

IV - JIT-compiler

47 JIT-compiler Intro

In the Brackets language a JIT-compiler can compile and run code at program run time.

The allowed code are math expressions, “if”, “else”, “endif” and jumps.

The “push/pull” and “load” opcodes are now included in the new experimental directory.

Now whole code blocks with variable read/write can be JIT-compiled. So you are not longer forced to do inline assembly! This is a big change!

Here the code between the labels “:jit_start” and “:jit_end” is compiled by the JIT-compiler.

47.1 Building the JIT-compiler and the VM

The JIT-compiler is now in its own repo on GitHub: “<https://github.com/koder77/llvm-jit-comp>”. Clone it and go into one of the directories: “stable” or “experimental”. The “experimental” directory contains the current AsmJIT library from 2026/05/01. I choose the “experimental” directory.

Go into a directory on your harddisk where you want store the repos.

Now in a shello do:

```
$ clone https://github.com/koder77/llvm.git
```

The same for the JIT-compiler:

```
$ git clone https://github.com/koder77/llvm-jit-comp.git
```

Now we want to build the X86-64 experimental JIT-compiler with the latest changes.

```
$ cd llvm-jit-comp/experimental/asmjit
```

Now run the build script:

```
$ ./build.sh
```

You should see this last lines:

```
[100%] Linking CXX shared library libasmjit-llvm.so
[100%] Built target asmjit
```

Install the AsmJIT library:

```
$ cd build
$ sudo make install
```

The LLVM AsmJIT library is now built. Copy “libasmjit-llvm.so” into the “~/llvm/bin” directory. So it is in the llvm search path.

Go into the x86-64 JIT-compiler directory:

```
$ cd ../../libjit
```

We build the JIT-compiler with:

```
$ ./make.sh
```

After this you should see:

```
$ ls
jit.cpo  jit.cpp  make.sh  make-win.sh  README.md  zerobuild.txt
```

At this point we are ready. Now copy the "asmjit" directory into the root of the "llvm" source code tree. And then the "libjit" directory too.

Go into the "llvm/vm" directory. Check that in "jit.h" the following is set:

```
// set to 1 to enable JIT-compiler build
#define JIT_COMPILER
```

If not then edit the file.

Finally we can build the LLVM with the JIT-compiler:

```
$ CC=clang CXX=clang++ zerobuild zerobuild-relro.txt force
```

Copy the "llvm" binary into the "llvm" directory in your "/home" directory:

```
cp llvm ~/llvm/bin/llvm-jit
```

Now everything is ready. Have fun!

47.2 Example

Here is an example which runs a double numbers math loop:

```
// double-benchm-jit-06-asm.ll com
//
//
#include <intr.llh>
(main func)
  (set int64 1 zero 0)
  (set int64 1 one 1)
  (set int64 1 loop 0)
  (set int64 1 maxloop 600000000Q)
  (set double 1 zerod 0.0)
  (set double 1 countd 1.0)
  (set double 1 xd 23.0)
  (set double 1 yd 42.0)
  (ASM)
  loada zero, 0, 0
  loada one, 0, 1
  loada loop, 0, 2
  loadd countd, 0, 3
  loada maxloop, 0, 4
  loadd xd, 0, 5
  loadd yd, 0, 6
  // load labels
  loadl :jit_start, 7
  loadl :jit_end, 8
  // get JIT-compiler time start:
  intr0 24, 0, 0, 0
  // run JIT-compiler:
  intr0 253, 7, 8, 0
  // get JIT-compiler time end:
  intr0 25, 20, 0, 0
  // print time:
  intr0 4, 20, 0, 0
  // print newline:
  intr0 7, 0, 0, 0
  // call JIT-compiled code:
  intr0 254, 0, 0, 0
  // jump to :next, over the JIT-compiled code block
```

```

    jmp :next
:jit_start
:loop
    // add xd
    addd 3, 5, 3
    addd 3, 5, 3
    addd 3, 5, 3
    addd 3, 5, 3
    addd 3, 5, 3
    addd 3, 5, 3
    addd 3, 5, 3
    // add yd
    addd 3, 6, 3
    addd 3, 6, 3
    addd 3, 6, 3
    addd 3, 6, 3
    addd 3, 6, 3
    addd 3, 6, 3
    // increase loop
    addi 2, 1, 2
    lseqi 2, 4, 10
:jit_end
    jmp 10, :loop
:next
    // print result
    intr0 5, 3, 0, 0
    // print newline
    intr0 7, 0, 0, 0
    // exit
    intr0 255, 0, 0, 0
    (ASM_END)
(funcend)

```

This program runs at a speed close to a C program with the same calculations. It is written in inline assembly.

48 Latest Changes

48.1 New linter flags - func args, remove

Now a variable can be marked as not valid, if it is unsafe to use them. For example it contains a file handle which is for an closed file.

```
// (remove foo)
```

This makes the variable “foo” not usable anymore.

There is now a new function flag:

```
// (func args S incP int64)
```

The “S” marks the function “incP” as strict. The function call arguments are marked as not valid after the function call. And all return variables must end with an “R”. They can not be used as a input variable for a strict function.

Here is an example:

```
// math-inc-strict.llcom
//
// contracts: precondition/postcondition demo
// Define function as strict. With the "S" flag.
// Only variables with an "R" ending are allowed as return values.
// They are forbidden as function call arguments.
//
// (func args S incP int64)
// (return args incP int64 int64)
(linter)

#include <intr-func.llh>

(main func)
  (set const-int64 1 zero 0)
  (set const-int64 1 one 1)
  (set int64 1 x 42)
  (set int64 1 incxR 0)
  (set int64 1 f 0)
  (set int64 1 retR 0)
  (set const-string s error_inc_str "inc: error!")

  (x :incP !) (retR incxR stpop)
  (((retR zero !=) f :=) f if)
    (error_inc_str :print_s !)
    (:print_n !)
    (one :exit !)
  (endif)

  (incxR :print_i !)
  (:print_n !)

  (zero :exit !)
(funcend)

(contracts-on)
(variable-local-only-on)

(incP func)
  #var ~ incP
  (set int64 1 num~ 0)
```

```

(set int64 1 ret~ 0)
(set int64 1 check~ 0)
(set const-int64 1 zero~ 0)
(set const-int64 1 one~ 1)
(set int64 1 f~ 0)
(set const-int64 1 int64-max~ 9223372036854775807)
(set const-string s overflow_str~ "inc: error number overflow!")
(set const-string s error_int_str~ "inc: error internal!")

(num~ stpop)

(precondition)
(((num~ int64-max~ ==) f~ :=) f~ if)
  (overflow_str~ :print_s !)
  (:print_n !)
  (zero~ one~ stpush) (return)
(endif)
(precondition-end)

(num~ + one~ ret~ :=)

(postcondition)
(ret~ - one~ check~ :=)
(((check~ num~ !=) f~ :=) f~ if)
  (error_int_str~ :print_s !)
  (:print_n !)
  (zero~ one~ stpush) (return)
(endif)
(postcondition-end)

(ret~ zero~ stpush) (return)
(funcend)

```

48.2 New linter flags - strict pointer

There is a new linter flag, if it is set then a pointer must be of “const-int64” variable type.

Such a pointer then can not be modified after it is set by the “pointer” function.

Here is the linter flag:

```
// (pointer strict on)
```


49 L1VMS

49.1 Intro

The new L1VM core with the task scheduler for preemptive multitasking is L1VMS. With it there can be virtual threads runned inside the OS POSIX threads.

The scheduler executes a set number of opcodes and then switches to another thread. NOTE: if one thread calls a blocking syscall then it will block the execution and can't execute another thread. You could put your blocking function in another POSIX thread and use the message module to exchange data between them.

The L1VMS multitasking runs about 25% slower than a normal L1VM POSIX thread core.

But now you can scale the virtual threads up to more threads as a CPU has cores.

And here are some another good news. The L1VMS core also did pass the CBMC code check. This time the number of checks is more than 29000! With some string function checks which can not be solved. But they have nothing to do with the L1VMS runtime. So it is one of the few projects which use CPPcheck, Infer, Valgrind and CBMC to make sure that there are no serious bugs in the code!

49.2 A multithreaded example

Here is an example program which starts two POSIX threads, each with two virtual threads.

So the task scheduler handles the two virtual threads:

```
// pthread-2.llcom
// threading test
#include <intr-func.llh>
#include <misc-macros.llh>
#include <math-const.llh>
(main func)
  (set int64 1 zero 0)
  (set int64 1 one 1)
  (set int64 1 two 2)
  (set int64 1 ret 0)
  (set int64 1 f 0)
  (set int64 1 thralab 0)
  (set int64 1 thrblab 0)
  (set int64 1 cpuid 0)
  (set string s msgthrstr "starting Threads...")
  (set int64 1 thrastart 0)
  (set int64 1 thrbstart 0)

  (:thra thralab loadl)
  (:thrb thrblab loadl)
  (thralab + zero thralab :=)
  (thrblab + zero thrblab :=)

  (cpuid :get_cpu !)
  (cpuid + zero cpuid :=)
  (((cpuid zero ==) f =) f if)
    (msgthrstr :print_s !)
    (:print_n !)

  (thralab :thread !)
  (thrblab :thread !)
(endif)

(:join !)

(zero :exit !)
```

```

(funcend)

(thra func)
  #var ~ thra

  (set int64 1 zero~ 0)
  (set int64 1 run~)
  (set int64 1 retcode~ 0)
  (set string s hellostr~ "Hello from a!")
  (set int64 1 thr1lab~ 0)
  (set int64 1 thr2lab~ 0)
  (set int64 1 vcpu~ 0)
  (set int64 1 del~ 1000)
  (set int64 1 f~ 0)

  (((thrastart zero~ ==) f~ =) f~ if)
  (one thrastart :=)

  (:thra1 thr1lab~ loadl)
  (:thra2 thr2lab~ loadl)

  (thr1lab~ + zero thr1lab~ :=)
  (thr2lab~ + zero thr2lab~ :=)

  (hellostr~ :print_s !)
  (:print_n !)

  (thr1lab~, vcpu~ :vcpu_RUN !)
  (del~ :detime !)
  (thr2lab~, vcpu~ :vcpu_RUN !)
  (del~ :detime !)

  (:wait_thra)
  (run~ :vcpu_JOIN !)
  (run~ + zero run~ :=)

  (((run~ zero~ ==) f~ =) f~ if+)
  (zero~ :threadexit !)
  (else)
    (:wait_thra jmp)
  (endif)
  (endif)
(funcend)

(thrb func)
  #var ~ thrb

  (set int64 1 zero~ 0)
  (set int64 1 run~)
  (set int64 1 retcode~ 0)
  (set string s hellostr~ "Hello from b!")
  (set int64 1 thr1lab~ 0)
  (set int64 1 thr2lab~ 0)
  (set int64 1 vcpu~ 0)
  (set int64 1 del~ 1000)
  (set int64 1 f~ 0)

  (((thrbstart zero~ ==) f~ =) f~ if)
  (one thrbstart :=)

```

```

(:thrb1 thr1lab~ loadl)
(:thrb2 thr2lab~ loadl)

(thr1lab~ + zero thr1lab~ :=)
(thr2lab~ + zero thr2lab~ :=)

(hellostr~ :print_s !)
(:print_n !)

(thr1lab~, vcpu~ :vcpu_RUN !)
(del~ :detime !)
(thr2lab~, vcpu~ :vcpu_RUN !)
(del~ :detime !)

(:wait_thrb)
(run~ :vcpu_JOIN !)
(run~ + zero run~ :=)

(((run~ zero~ ==) f~ =) f~ if+)
  (zero~ :threadexit !)
(else)
  (:wait_thrb jmp)
(endif)
(endif)
(funcend)

// vcpu threads
(thra1 func)
  #var ~ thra1
  (set string s hellostr~ "thra1 Hello world: CPU ")
  (set string s countstr~ " count: ")
  (set int64 1 cpu~ 0)
  (set int64 1 i~ 0)
  (set int64 1 max~ 1000000)
  (set int64 1 f~ 0)
  (zero i~ :=)
  //(cpu~ stpop)
  (:print_a1_loop)
  (hellostr~ :print_s !)
  (cpu~ :print_i !)
  (countstr~ :print_s !)
  (i~ :print_i !)
  (:print_n !)
  (i~ + one i~ :=)
  (((i~ max~ <) f~ :=) f~ if)
    (:print_a1_loop jmp)
  (endif)
  (:vcpu_EXIT !)
(funcend)

(thra2 func)
  #var ~ thra2
  (set string s hellostr~ "thra2 Hello world: CPU ")
  (set string s countstr~ " count: ")
  (set int64 1 cpu~ 0)
  (set int64 1 i~ 0)
  (set int64 1 max~ 1000000)
  (set int64 1 f~ 0)
  (zero i~ :=)
  //(cpu~ stpop)

```

```

(:print _a2_loop)
(hellostr~ :print _s !)
(cpu~ :print _i !)
(countstr~ :print _s !)
(i~ :print _i !)
(:print _n !)
(i~ + one i~ :=)
(((i~ max~ <) f~ :=) f~ if)
  (:print _a2_loop jmp)
(endif)
(:vcpu_EXIT !)
(funcend)

(thrb1 func)
#var ~ thrb1
(set string s hellostr~ "thrb1 Hello world: CPU ")
(set string s countstr~ " count: ")
(set int64 1 cpu~ 0)
(set int64 1 i~ 0)
(set int64 1 max~ 1000000)
(set int64 1 f~ 0)
(zero i~ :=)
//(cpu~ stpop)
(:print _b1_loop)
(hellostr~ :print _s !)
(cpu~ :print _i !)
(countstr~ :print _s !)
(i~ :print _i !)
(:print _n !)
(i~ + one i~ :=)
(((i~ max~ <) f~ :=) f~ if)
  (:print _b1_loop jmp)
(endif)
(:vcpu_EXIT !)
(funcend)

(thrb2 func)
#var ~ thrb2
(set string s hellostr~ "thrb2 Hello world: CPU ")
(set string s countstr~ " count: ")
(set int64 1 cpu~ 0)
(set int64 1 i~ 0)
(set int64 1 max~ 1000000)
(set int64 1 f~ 0)
(zero i~ :=)
//(cpu~ stpop)
(:print _b2_loop)
(hellostr~ :print _s !)
(cpu~ :print _i !)
(countstr~ :print _s !)
(i~ :print _i !)
(:print _n !)
(i~ + one i~ :=)
(((i~ max~ <) f~ :=) f~ if)
  (:print _b2_loop jmp)
(endif)
(:vcpu_EXIT !)
(funcend)

```

50 Linter - function prototypes

50.1 Intro

The linter can be used to check if in a function call the argument types match. The same can be done by the “push” return values. In the “linter-autogen” directory is a script “make-lint-func.sh”. With this script a function prototype “.llh” file can be created. It reads the module C source code and the “.llh” module header file. The generated header file contains the function prototypes for use with the linter.

50.2 An example

In the C source code of a module are often multiple return paths. With different variable pushes.

The “make-lint-func.sh” script uses two C tools to do the work. If there are multiple return paths you need to set a special comment flag:

```
// return-start  
push code...  
// return-end
```

Here is a part of crypto.c:

```
if (crypto_secretbox_open_easy (&data[out_address], &data[in_address], cipher_text_len,  
&data[nonce_address], &data[key_address]) != 0)  
{  
    // error on decrypt!!!  
    sp = stpushi (1, sp, sp_bottom);  
    if (sp == NULL)  
    {  
        // error  
        printf ("encrypt_sodium: ERROR: stack corrupt!\n");  
        return (NULL);  
    }  
}  
}  
  
// return-start  
sp = stpushi (0, sp, sp_bottom);  
if (sp == NULL)  
{  
    // error  
    printf ("encrypt_sodium: ERROR: stack corrupt!\n");  
    return (NULL);  
}  
  
return (sp);  
// return-end  
}
```

I hope this makes it easy to understand. For the other flags see the source code of the two tools!

51 References

- [1] “The Structure and Performance of Efficient Interpreters” (11/2003) - page 1
M. Anton Ertl - Institut für Computersprachen, TU Wien, Argentinertstraße 8, A-1040 Wien
David Gregg - Department of Computer Science, Trinity College, Dublin 2, Ireland
- [2] “The Structure and Performance of Efficient Interpreters” (11/2003) - page 2
- [3] “Virtual Machine Showdown: Stack Versus Registers” (2008)
Yunhe Shi, David Gregg, Andrew Beatty, Department of Computer Science University of Dublin,
Trinity College Dublin 2, Ireland
M. Anton Ertl - Institut für Computersprachen, TU Wien, Argentinertstraße 8, A-1040 Wien